WNDRVR
SUPPORT NETWORK

VXWORKS 7 C11, C++14, C++17, AND COMPILER MIGRATION REFERENCE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 C11, C++14, C++17, and Compiler Migration Reference, SR0650

2 July 2020

1. INTRODUCTION

1. Introduction

VxWorks 7® introduces *ISO/IEC 9899:2011* (aka **C11**), *ISO/IEC 14882:2014* (aka **C++14**), and ISO/IEC 14882:2017 (aka **C++17**) conformant standard **C** and **C++** libraries for both the kernel and user-space environments. Prior to the update of the **C** and **C++** standard libraries, the kernel environment provided **C89** and **C++03** standard libraries, and the RTP environment provided **C99** and **C++03** (or **C++11** with GNU compiler) standard libraries.

Given that the **C** standard library previously provided in the kernel environment had numerous deviations from the **C89** standard, there are several compatibility issues associated with the standard **C** library upgrade in the kernel. There are also some compatibility issues associated with the standard **C** and **C++** library upgrades in user-space, but to a significantly lesser degree than with the changes in the kernel environment.

Concurrent with the **C** and **C++** standard library upgrade, the Wind River Compiler and the GNU compiler are being replaced with the Clang compiler for 32-bit ARM, and both 32-bit and 64-bit Intel. The Clang compiler was the only compiler supported when 64-bit ARM, 32-bit RISC-V, and 64-bit RISC-V support were initially added to VxWorks 7. The Wind River Diab Compiler is replaced by a stock GCC.

The purpose of this document is to list any compatibility issues associated with the standard **C** and **C++** library upgrade along with the switch to the Clang compiler (for ARM and Intel) and to a stock GCC compiler (for PowerPC). This document serves as an aid to existing customers to determine the scope of the impact of upgrading to the release of VxWorks 7 that introduces the **C11** and **C++14/C++17** conformant standard libraries, along with the switch to the Clang compiler.

This document does not provide in great detail the new definitions/interfaces available with **C11** versus **C89** or **C99**, nor does it provide in great detail the new definitions/interfaces available with **C++14/C++17** versus **C++03** or **C++11**. The only information regarding new definitions/interfaces introduced by **C11** or **C++14/C++17** is a result of listing the definitions provided in certain standard header files, such as time.h, string.h, and so forth.

2. REFERENCES

1. References

- ISO/IEC 14882:2014 Specification Document (must be purchased)
  https://www.iso.org/standard/64029.html
- ISO/IEC 9899:2011 Specification Document (must be purchased)
  https://www.iso.org/standard/57853.html
- clang: a C language family frontend for LLVM
  https://docs.windriver.com/category/dt-llvm_compiler

3. THE COMPILATION ENVIRONMENT

1. The Compilation Environment

This section describes the compilation environment.

In both the kernel and RTP compilation environments, the inclusion of header files specified by *ISO/IEC 9899:2011* (aka **C11**), *ISO/IEC 14882:2014* (aka **C++14**), and *ISO/IEC 14882:2017* (aka **C++17**) results in definitions that are described by those specifications, as well as a subset of definitions from the POSIX.1-2008 and X/Open System Interfaces specifications.

Previously, "standard" header files, such as **time.h**, contained VxWorks-specific interfaces. With the introduction of **C11**, **C++14** and **C++17** standard libraries, the default is to continue providing most of these VxWorks-specific interfaces. Any effects that occurred when setting the **_VXWORKS_COMPATIBILITY_MODE** macro are not preserved, nor are any function signatures that deviated from the **C11** or **C++14/C++17** specifications. Any VxWorks-specific interfaces that are no longer provided are indicated in the appropriate section of this document.

To suppress the visibility of any of these extensions to an application, the appropriate feature test macro needs to be defined by the application source file before including any of the header files defined by **C11** or **C++14/C++17**.

As an example, the POSIX.1-2008 specification introduces the tzset( ) interface to **time.h**. See the following OpenGroup page for more information on this extension:

http://pubs.opengroup.org/onlinepubs/9699919799/functions/tzset.html

Furthermore, the X/Open System Interfaces specification defines a pair of global variables, *'daylight'* and *'timezone'*, which are updated when calling tzset( ). By default, an application source file that includes **time.h** shall have access to the tzset( ) function prototype, and the declarations of the *'tzname'*, *'daylight'*, and *'timezone'* variables. For an application source file that utilizes the tzset( ) interface, but does not access the *'daylight'* and *'timezone'* global variables, nor wants these symbols to pollute the namespace, the **_POSIX_C_SOURCE** macro should be defined before including **time.h:**

```
#define _POSIX_C_SOURCE
#include <time.h>
```

Another example involves the following pair of VxWorks-specific interfaces available through **string.h**:

```
size_t   strlcat (char *, const char *, size_t);
size_t   strlcpy (char *, const char *, size_t);
```

By default, an application source file that includes **string.h** shall have access to the strlcat( ) and strlcpy( ) function prototypes. If an application needs to suppress the definition of these function prototypes, the following should be performed before the inclusion of **string.h**:

```
#define _POSIX_C_SOURCE
#include <string.h>
```

Or:

```
#define _XOPEN_SOURCE
#include <string.h>
```


WNDRVR

2. Summary of Extensions

As mentioned above, the inclusion of header files specified by **C11** and **C++14/C++17** results in definitions that are described by those specifications, as well as a **subset** of definitions from the POSIX.1-2008 and X/Open System Interfaces specifications, along with some VxWorks native interfaces. This section will provide a summary of the extensions provided through headers files from the VxWorks 7 standard C library layer.

POSIX.1-2008 Extensions on page 4 The following interfaces are provided through **time.h**:
X/Open System Interface (XSI) Extensions on page 5 The header file **search.h** is not specified by **C11**, however it is provided by the VxWorks 7 standard **C** library for the RTP environment. The following interfaces are provided through **search.h**:
BSD Extensions on page 5 The following interfaces are provided through **string.h:**
VxWorks Native Interface Extensions on page 5 The following interface is provided through **time.h** in the kernel environment:

2.1. POSIX.1-2008 Extensions

The following interfaces are provided through **time.h**:

```
int           clock_gettime         (clockid_t clock_id, struct timespec * tp);
int           clock_settime         (clockid_t clock_id, const struct timespec * tp);
int           clock_getres          (clockid_t clock_id, struct timespec * res);
int           clock_nanosleep       (clockid_t clock_id, int flags,
                                     const struct timespec * rqtp,
                                     struct timespec * rmtp);

int           timer_create          (clockid_t clock_id,
                                     struct sigevent *_Restrict evp,
                                     timer_t *_Restrict ptimer);
int           timer_delete          (timer_t timerid);
int           timer_gettime         (timer_t timerid, struct itimerspec * value);
int           timer_settime         (timer_t timerid, int flags,
                                     const struct itimerspec *_Restrict value,
                                     struct itimerspec *_Restrict ovalue);
int           timer_getoverrun      (timer_t timerid);
int           nanosleep             (const struct timespec * rqtp,
                                     struct timespec * rmtp);

void          tzset                 (void);
char *        asctime_r             (const struct tm *, char *);
char *        ctime_r               (const time_t *, char *);
struct tm *   gmtime_r              (const time_t *, struct tm *);
struct tm *   localtime_r           (const time_t *, struct tm *);
```

The following interfaces are provided through **string.h**:

```
int       strerror_r      (int, char *, size_t);
```

The following interfaces are provided through **stdlib.h**:

```
int       setenv          (const char *, const char *, int);
int       unsetenv        (const char *);
```

Parent topic: Summary of Extensions on page 4

2.2. X/Open System Interface (XSI) Extensions

The header file **search.h** is not specified by **C11**, however it is provided by the VxWorks 7 standard **C** library for the RTP environment. The following interfaces are provided through **search.h**:

```
ENTRY   *hsearch          (ENTRY, ACTION);
int      hcreate          (unsigned int);
void     hdestroy         (void);
void *   lsearch          (const void *, const void *,
                          unsigned int *, int, int (*)());
void *   lfind            (const void *, const void *, unsigned int *,
                          int, int (*)());
void *   tsearch          (const void *, void **, int (*)());
void *   tfind            (const void *, void *const *, int (*)());
void *   tdelete          (const void *, void **, int (*)());
void     twalk            (void *, void (*)());
```

The header file **strings.h** is not specified by **C11**, however it is provided by the VxWorks 7 standard **C** library. The following interfaces are provided through **strings.h** and **string.h**:

```
int      bcmp                (const void *, const void *, size_t);
void     bcopy               (const void *, void *, size_t);
void     bzero               (void *, size_t);
char *   index               (const char *, int);
char *   rindex              (const char *, int);

int      strcasecmp          (const char *, const char *);
int      strncasecmp         (const char *, const char *, size_t);
```

The following interfaces are provided through **ctype.h**:

```
int     isascii         (int);
int     toascii         (int);
```

The following interfaces are provided through **stdlib.h**:

```
int     putenv                  (char *);
```

Parent topic: Summary of Extensions on page 4

2.3. BSD Extensions

The following interfaces are provided through **string.h:**

```
size_t  strlcpy         (char *, const char *, size_t);
size_t  strlcat         (char *, const char *, size_t);
```

Parent topic: Summary of Extensions on page 4

2.4. VxWorks Native Interface Extensions

The following interface is provided through **time.h** in the kernel environment:

```
int     clock_adjtime         (clockid_t clock_id, const struct timespec * delta,
                               struct timespec * olddelta);

int     timer_show            (timer_t timerid, int verbose);
STATUS  timer_modify          (timer_t timerId, struct sigevent * pSigev);
```

The following interfaces are provided through **time.h** in both the kernel and RTP environments:

```
Int       timer_connect       (timer_t timerid, TIMER_CONNECT_FUNC routine,
                               _Vx_usr_arg_t arg);
int       timer_cancel        (timer_t timerid);
timer_t   timer_open          (const char * name, int mode, clockid_t clockId,
                               struct sigevent * evp, void * context);
STATUS    timer_close         (timer_t timerId);
STATUS    timer_unlink        (const char * name);
```

The header file strings.h is not specified by **C11**, however it is provided by the VxWorks 7 standard **C** library. The following interfaces are provided through **strings.h** and string.h:

```
void    binvert                 (void *, size_t);
void    bswap                   (void *, void *, size_t);
void    uswab                   (const void *, void *, size_t);
void    bcopyBytes              (const void *, void *, size_t);
void    bcopyWords              (const void *, void *, size_t);
void    bcopyLongs              (const void *, void *, size_t);
void    bcopyQuads              (const void *, void *, size_t);
void    bfill                   (void *, size_t, int);
void    bfillBytes              (void *, size_t, int);
```

Parent topic: Summary of Extensions on page 4

4. PROJECT CONFIGURATION

1. VxWorks Source Build (VSB) Configuration

Previously, the standard **C** library was provided by the **LIBC_KERNEL** and **LIBC_USER VSB** layers for the kernel and user-space environments, respectively. Both of these layers are sublayers of the **LANG_LIB_LIBC** layer.

The standard **C++** library was provided by the **CPLUS_KERNEL** and **CPLUS_USER** layers for the kernel and user-space environments, respectively. Both of these layers are sublayers of the **LANG_LIB_CPLUS** layer.

As a result of the **C11** standard library upgrade, the standard **C** library is provided by the **LIBC_STD** layer for both the kernel and user-space environments. Support for future **C** language specifications will still be provided by the **LIBC_STD** layer.

As a result of the **C++14/C++17** standard library upgrade, the standard **C++** library is provided by the **LIBCPLUS_STD** layer for both the kernel and user-space environments. Support for future **C++** language specifications will still be provided by the **LIBCPLUS_STD** layer.

2. VxWorks Image Project (VIP) Configuration

Previously, the standard **C** library was provided by the following set of components:

INCLUDE_ANSI_ABORT
INCLUDE_ANSI_ABS
INCLUDE_ANSI_ASSERT
INCLUDE_ANSI_ATOF
INCLUDE_ANSI_BSEARCH
INCLUDE_ANSI_CTYPE
INCLUDE_ANSI_LOCALE
INCLUDE_ANSI_LONGJMP
INCLUDE_ANSI_MATH
INCLUDE_ANSI_MEMCHR
INCLUDE_ANSI_MEMCMP
INCLUDE_ANSI_MEMCPY
INCLUDE_ANSI_MEMMOVE
INCLUDE_ANSI_MEMSET
INCLUDE_ANSI_STDIO
INCLUDE_ANSI_STDIO_EXTRA
INCLUDE_ANSI_STDLIB
INCLUDE_ANSI_STDLIB_NUMBERS
INCLUDE_ANSI_STRCASECMP
INCLUDE_ANSI_STRCAT
INCLUDE_ANSI_STRCMP
INCLUDE_ANSI_STRCPY
INCLUDE_ANSI_STRDUP
INCLUDE_ANSI_STRERROR
INCLUDE_ANSI_STRING
INCLUDE_ANSI_STRLCAT
INCLUDE_ANSI_STRLCPY
INCLUDE_ANSI_STRLEN
INCLUDE_ANSI_STRNCASECMP
INCLUDE_ANSI_STRNCAT
INCLUDE_ANSI_STRNCMP
INCLUDE_ANSI_STRNCPY

INCLUDE_ANSI_STRTOD
INCLUDE_ANSI_TIME

As a result of the **C11** standard library upgrade, a component exists for each header file specified by *ISO/IEC 9899:2011*. For example, to include all of the interfaces defined in **stdio.h**, the **INCLUDE_ANSI_STDIO** component should be included in a VIP. Thus the following components reside under the **FOLDER_ANSI** folder:

INCLUDE_ANSI_ASSERT
INCLUDE_ANSI_COMPLEX
INCLUDE_ANSI_CTYPE
INCLUDE_ANSI_ERRNO
INCLUDE_ANSI_FENV
INCLUDE_ANSI_FLOAT
INCLUDE_ANSI_INTTYPES
INCLUDE_ANSI_ISO646
INCLUDE_ANSI_LIMITS
INCLUDE_ANSI_LOCALE
INCLUDE_ANSI_MATH
INCLUDE_ANSI_SETJMP
INCLUDE_ANSI_SIGNAL
INCLUDE_ANSI_STDALIGN
INCLUDE_ANSI_STDARG
INCLUDE_ANSI_STDATOMIC
INCLUDE_ANSI_STDBOOL
INCLUDE_ANSI_STDDEF
INCLUDE_ANSI_STDINT
INCLUDE_ANSI_STDIO
INCLUDE_ANSI_STDLIB
INCLUDE_ANSI_STDNORETURN
INCLUDE_ANSI_STRING
INCLUDE_ANSI_TGMATH
INCLUDE_ANSI_THREADS
INCLUDE_ANSI_TIME
INCLUDE_ANSI_UCHAR
INCLUDE_ANSI_WCHAR
INCLUDE_ANSI_WCTYPE

The **INCLUDE_ANSI_LONGJMP** component has been renamed to **INCLUDE_ANSI_SETJMP** to match the corresponding header file name; **setjmp.h**. Any component that did not map to a header file specified by *ISO/IEC 9899:2011*, e.g. **INCLUDE_ANSI_STRCPY**, has been removed.

Previously, the **INCLUDE_STRING_SEARCH** component provided the memmem( ) and memimem( ) interfaces. As a result of the **C11** standard library upgrade, the memmem( ) and memimem( ) interfaces are provided by the **INCLUDE_ANSI_STRING** component, and the **INCLUDE_STRING_SEARCH** component is deprecated.

In addition to the above list, the **INCLUDE_LIBC_STD** component has been introduced to include the entire standard **C** library into a VIP.

Previously, the **INCLUDE_CPLUS_IOSTREAMS** component was provided to include the entire standard **C++** library into the VIP. As a result of the **C++14/C++17** standard library upgrade, the **INCLUDE_CPLUS_IOSTREAMS** is deprecated. Instead, the **INCLUDE_CPLUS_STDLIB** component is used to include the entire standard **C++** library into the VIP.

WNDRVR

5. KERNEL ENVIRONMENT: C11 COMPATIBILITY

1. Environment Variables

Previously the following function prototypes were available through **stdlib.h**:

```
char *   getenv       (const char *__name);
int      getenv_s     (size_t * len, char * value, size_t maxsize,
                       const char * name);
int      setenv        (const char *, const char *, int);
int      unsetenv      (const char *);
```

In addition, previously, the following function prototypes were available through **envLib.h**:

```
STATUS   putenv         (char * pEnvString);
```

As a result of the **C11** standard library upgrade, the following environment variable handling function prototypes are available through **stdlib.h** :

```
errno_t       getenv_s      (size_t *_Restrict, char *_Restrict, rsize_t,
                              const char *_Restrict);
char *         getenv        (const char *);
```

The following function prototype is also available through **stdlib.h** but can be suppressed through the **_POSIX_C_SOURCE** feature test macro:

```
int putenv (char *string);
```

The following function prototypes are also available through **stdlib.h** but can be suppressed through the **_ISO_C_SOURCE** feature test macro:

```
int           setenv        (const char *, const char *, int);
int           unsetenv      (const char *);
```

Although the signature of getenv_s( ) and putenv( ) has changed as shown in bold, the base type of **rsize_t** and **size_t** are identical. Similarly, the base type of **errno_t** is 'int'. For VxWorks 32-bit, the following definitions apply:

```
typedef unsigned int      size_t;
typedef unsigned int      rsize_t;
typedef int               errno_t;
```

For VxWorks 64-bit, the following definitions apply:

```
typedef unsigned long      size_t;
typedef unsigned long      rsize_t;
typedef int                errno_t;
```

Note: The following VxWorks native interfaces related to environment variable handling are still supported through **envLib.h**

```
STATUS   envPrivateCreate     (TASK_ID taskId, _Vx_usr_arg_t envSource);
STATUS   envPrivateDestroy    (TASK_ID taskId);
char **  envGet               (TASK_ID taskId);
void     envShow              (TASK_ID taskId);
```

Summary of Environment Variables Changes on page 10

1.1. Summary of Environment Variables Changes

- The signature of getenv_s( ) and putenv( ) has changed.
- The putenv( ) function prototype is available through **stdlib.h** instead of **envLib.h**.
- The setenv( ) and unsetenv( ) function prototypes are available through **stdlib.h**.

Parent topic: Environment Variables on page 9

2. Date and Time Interfaces: time.h

Previously, the following interfaces were provided in **time.h**:

```
char *          asctime        (const struct tm *_tptr);
clock_t         clock          (void);
char *          ctime          (const time_t *_cal);
double          difftime       (time_t _t1, time_t _t0);
struct tm *     gmtime         (const time_t *_tod);
struct tm *     localtime      (const time_t *_tod);
time_t          mktime         (struct tm *_tptr);
size_t          strftime       (char *_s, size_t _n, const char *_format,
                                const struct tm *_tptr);
time_t          time           (time_t *_tod);
```

In addition, **time.h** unconditionally included **clockLib.h** and **timerLib.h**. The interfaces provided by **clockLib.h** and **timerLib.h** are **POSIX.1-2008** extensions. Thus the following interfaces were also available through the inclusion of **time.h**:

```
int     timer_create          (clockid_t clock_id, struct sigevent * evp,
                               timer_t * ptimer);
int     timer_delete          (timer_t timerid);
int     timer_gettime         (timer_t timerid, struct itimerspec * value);
int     timer_settime         (timer_t timerid, int flags,
                               const struct itimerspec * value,
                               struct itimerspec * ovalue);
int     timer_getoverrun      (timer_t timerid);
int     timer_connect         (timer_t timerid, TIMER_CONNECT_FUNC routine,
                               _Vx_usr_arg_t arg);
int     timer_cancel          (timer_t timerid);
timer_t timer_open            (const char * name, int mode, clockid_t clockId,
                               struct sigevent * evp, void * context);
STATUS  timer_close           (timer_t timerId);
```

```
STATUS  timer_unlink          (const char * name);

int     timer_show            (timer_t timerid, int verbose);
STATUS  timer_modify          (timer_t timerId, struct sigevent * pSigev);

int     nanosleep             (const struct timespec * rqtp,
                               struct timespec * rmtp);

int     clock_gettime         (clockid_t clock_id, struct timespec * tp);
int     clock_settime         (clockid_t clock_id, const struct timespec * tp);
int     clock_adjtime         (clockid_t clock_id, const struct timespec * delta,
                               struct timespec * olddelta);
int     clock_getres          (clockid_t clock_id, struct timespec * res);

int     clock_nanosleep       (clockid_t clock_id, int flags,
                               const struct timespec * rqtp,
                               struct timespec * rmtp);
```

Previously, the inclusion of **time.h** provided definitions for the following **POSIX.1-2008** extensions conditionalized on the **_EXTENSION_POSIX_REENTRANT** macro. The **_EXTENSION_POSIX_REENTRANT** macro was defined to '1' in **vxAnsi.h**:

```
char *        asctime_r     (const struct tm *_tm, char *_buffer);
char *        ctime_r       (const time_t *_cal, char *_buffer);
struct tm *   gmtime_r      (const time_t *_tod,  struct tm *_result);
struct tm *   localtime_r   (const time_t *_tod, struct tm *_result);
```

Previously, the inclusion of **time.h** provided the following VxWorks specific variants of asctime_r( ), ctime_r( ), gmtime_r( ), and localtime_r( ) if the **_VXWORKS_COMPATIBILITY_MODE** macro was defined by the including source file:

```
int       asctime_r     (const struct tm *_tm, char *_buffer, size_t *_buflen);
char *     ctime_r       (const time_t *_cal, char *_buffer, size_t *_buflen);
int        gmtime_r      (const time_t *_tod, struct tm *_result);
int       localtime_r   (const time_t *_tod, struct tm *_result);
```

The differences between the **POSIX.1-2008** and VxWorks specific definitions of the interfaces have been bolded.

As a result of the **C11** standard library upgrade, the following function prototypes are provided in **time.h** :

```
time_t      time              (time_t *);
int         timespec_get      (struct timespec *ts, int base);
char *      asctime           (const struct tm *);
clock_t     clock             (void);
char *      ctime             (const time_t *);
double      difftime          (time_t, time_t);
struct tm * gmtime            (const time_t *);
struct tm * localtime         (const time_t *);
time_t      mktime            (struct tm *);
size_t      strftime          (char *_Restrict, size_t, const char *_Restrict,
                               const struct tm *_Restrict);
errno_t     asctime_s         (char *, rsize_t, const struct tm *);
errno_t     ctime_s           (char *, rsize_t, const time_t *);
struct tm * gmtime_s          (const time_t *_Restrict, struct tm *_Restrict);
struct tm * localtime_s       (const time_t *_Restrict, struct tm *_Restrict);
void        tzset             (void);
char *      asctime_r         (const struct tm *, char *);
```

```
char *      ctime_r           (const time_t *, char *);
struct tm * gmtime_r          (const time_t *, struct tm *);
struct tm * localtime_r       (const time_t *, struct tm *);

char *      tzname[2];
```

time.h continues to include **clockLib.h** and **timerLib.h**. These can be suppressed through the **_ISO_C_SOURCE** feature test macro.

In addition, The following definitions can also be suppressed though the **_ISO_C_SOURCE** feature test macro:

```
void             tzset        (void);
char *           asctime_r    (const struct tm *, char *);
char *           ctime_r      (const time_t *, char *);
struct tm *      gmtime_r     (const time_t *, struct tm *);
struct tm *      localtime_r  (const time_t *, struct tm *);
char *           tzname[2];
```

The following X/Open Source Interface extensions related to tzset( ) are also available but can be suppressed through the **_POSIX_C_SOURCE** feature test macro:

```
long          timezone;
int           daylight;
```

As a result of the **C11** standard upgrade, a variety of internal time related APIs have been eliminated:

```
int      __daysSinceEpoch    ();
int      __getTime           ();
int      __getDstInfo        ();
int      __getDstInfoSub     ();
void     __getZoneInfo       ();
```

The above function prototypes were provided in the **timeP.h** header file and thus are considered private, i.e. standards conforming applications should never include private VxWorks header files, .e.g header files whose name are suffixed with a capital letter "P".

Summary of Changes to time.h on page 12

2.1. Summary of Changes to time.h

- The **_VXWORKS_COMPATIBILITY_MODE** macro is no longer supported, i.e. the VxWorks specific variants of asctimer_r( ), ctime_r( ), gmtime_r( ), and localtime_r( ) are no longer supported.

Parent topic: Date and Time Interfaces: time.h on page 10

3. String Handling Interfaces: strings.h and string.h

Previously, the following interfaces were provided in **strings.h** :

```
int    strcasecmp       (const char *, const char *);
int    strncasecmp      (const char *, const char *, size_t);
```


WNDRVR

Previously, the following interfaces were provided in **string.h**:

```
void *  memchr      (const void *__s, int __c, size_t __n);
int     memcmp      (const void *__s1, const void *__s2, size_t __n);
void *  memcpy      (void *__s1, const void *__s2, size_t __n);
void *  memmove     (void *__s1, const void *__s2, size_t __n);
void *  memset      (void *__s, int __c, size_t __n);
errno_t memset_s    (void *dest, rsize_t destsz, int ch, rsize_t count);

char *  strcat      (char *__s1, const char *__s2);
char *  strchr      (const char *__s, int __c);
int     strcmp      (const char *__s1, const char *__s2);
int     strcoll     (const char *__s1, const char *__s2);
char *  strcpy      (char *__s1, const char *__s2);
size_t  strcspn     (const char *__s1, const char *__s2);
char *  strdup      (const char * str);
size_t  strlen      (const char *__s);
char *  strncat     (char *__s1, const char *__s2, size_t __n);
int     strncmp     (const char *__s1, const char *__s2, size_t __n);
char *  strncpy     (char *__s1, const char *__s2, size_t __n);
char *  strpbrk     (const char *__s1, const char *__s2);
char *  strrchr     (const char *__s, int __c);
size_t  strspn      (const char *__s1, const char *__s2);
char *  strstr      (const char *__s1, const char *__s2);
char *  strtok      (char *__s, const char *__sep);
size_t  strxfrm     (char *__s1, const char *__s2, size_t __n);
char *  strtok_r    (char *__s, const char *__sep, char **__ppLast);
char *  strerror    (int __errcode);
int     strerror_r  (int __errcode, char *__buf);

void    bcopy       (const void *source, void *dest, size_t nbytes);
void    bcopyBytes  (const void *source, void *dest, size_t nbytes);
void    bcopyWords  (const void *source, void *dest, size_t nwords);
void    bcopyLongs  (const void *source, void *dest, size_t nlongs);
void    bcopyQuads  (const void *, void *, size_t);
void    bfill       (void *buf, size_t nbytes, int ch);
void    bfillBytes  (void *buf, size_t nbytes, int ch);
void    bzero       (void *buffer, size_t nbytes);
int     bcmp        (const void *buf1, const void *buf2, size_t nbytes);
void    binvert     (void *buf, size_t nbytes);
void    bswap       (void *buf1, void *buf2, size_t nbytes);
void    uswab       (const void *source, void *destination, size_t nbytes);

char *  index       (const char *s, int c);
char *  rindex      (const char *s, int c);

size_t  strlcat     (char *dst, const char *src, size_t siz);
size_t  strlcpy     (char *dst, const char *src, size_t siz);

void *  memmem     (const void * haystack, size_t haystackLen,
                    const void * needle, size_t needleLen);
void *  memimem    (const void * haystack, size_t haystackLen,
```

The include file string.h is specified by **C11** but **strings.h** is an X/Open System Interfaces extension. As a result of the **C11** standard library upgrade, the function prototypes listed below are available through **strings.h**. These prototypes can be suppressed though the **_ISO_C_SOURCE** feature test macro.

```
int      bcmp        (const void *, const void *, size_t);
void    bcopy       (const void *, void *, size_t);
void    bzero       (void *, size_t);
char *  index       (const char *, int);
char *  rindex      (const char *, int);

int     strcasecmp  (const char *, const char *);
int     strncasecmp (const char *, const char *, size_t);
```

The following VxWorks-specific function prototypes are also available through strings.h, but only if the **_POSIX_C_SOURCE**, and **_POSIX_AEP_RT_CONTROLLER_C_SOURCE**, and **_XOPEN_SOURCE**, and **_ISO_C_SOURCE** feature test macros are **NOT** defined by the including source file:

```
void    binvert     (void *, size_t);
void    bswap       (void *, void *, size_t);
void    uswab       (const void *, void *, size_t);
void    bcopyBytes  (const void *, void *, size_t);
void    bcopyWords  (const void *, void *, size_t);
void    bcopyLongs  (const void *, void *, size_t);
void    bcopyQuads  (const void *, void *, size_t);
void    bfill       (void *, size_t, int);
void    bfillBytes  (void *, size_t, int);

void *  memmem      (const void * haystack, size_t haystackLen,
                     const void * needle, size_t needleLen);
void *  memimem     (const void * haystack, size_t haystackLen,
                     const void * needle, size_t needleLen);
```

As a result of the **C11** standard library upgrade, the following function prototypes are available through **string.h**:

```
int        memcmp          (const void *, const void *, size_t);
void *     memcpy          (void *_Restrict, const void *_Restrict, size_t);
void *     memset          (void *, int, size_t);
char *     strcat          (char *_Restrict, const char *_Restrict);
int        strcmp          (const char *, const char *);
char *     strcpy          (char *_Restrict, const char *_Restrict);
size_t     strlen          (const char *);

void *     memmove         (void *, const void *, size_t);
int        strcoll         (const char *, const char *);
size_t     strcspn         (const char *, const char *);
char *     strerror        (int);
char *     strncat         (char *_Restrict, const char *_Restrict, size_t);
int        strncmp         (const char *, const char *, size_t);
char *     strncpy         (char *_Restrict, const char *_Restrict, size_t);
size_t     strspn          (const char *, const char *);
char *     strtok          (char *_Restrict, const char *_Restrict);
size_t     strxfrm         (char *_Restrict, const char *_Restrict, size_t);
char *     strdup          (const char *);
int        strcasecmp      (const char *, const char *);
int        strncasecmp     (const char *, const char *, size_t);
char *     strtok_r        (char *, const char *, char **);
char *     strchr          (const char *, int);
char *     strpbrk         (const char *, const char *);
char *     strrchr         (const char *, int);
char *     strstr          (const char *, const char *);
```

```
void *     memchr          (const void *, int, size_t);

errno_t    memcpy_s        (void *_Restrict, rsize_t, const void *_Restrict,
                            rsize_t);
errno_t    memmove_s       (void *, rsize_t, const void *, rsize_t);
errno_t    strcpy_s        (char *_Restrict, rsize_t, const char *_Restrict);
errno_t    strncpy_s       (char *_Restrict, rsize_t, const char *_Restrict,  rsize_t);
errno_t    strcat_s        (char *_Restrict, rsize_t, const char *_Restrict);
errno_t    strncat_s       (char *_Restrict, rsize_t, const char *_Restrict,
                            rsize_t);
char *     strtok_s        (char *_Restrict, rsize_t *_Restrict,
                            const char *_Restrict, char **_Restrict);

errno_t    memset_s        (void *, rsize_t, int, rsize_t);
errno_t    strerror_s      (char *, rsize_t, errno_t);
size_t     strerrorlen_s   (errno_t);
size_t     strnlen_s       (const char *_Restrict, size_t);
int        strerror_r      (int, char *, size_t);

int        bcmp            (const void *, const void *, size_t);
void       bcopy           (const void *, void *, size_t);
void       bzero           (void *, size_t);
char *     index           (const char *, int);
char *     rindex          (const char *, int);
```

The include file string.h is specified by **C11** but **string.h** is an X/Open System Interfaces extension. As a result of the **C11** standard library upgrade, the function prototypes listed below are available through **string.h**. These prototypes can be suppressed though the **_ISO_C_SOURCE** feature test macro.

```
int      bcmp        (const void *, const void *, size_t);
void    bcopy       (const void *, void *, size_t);
void    bzero       (void *, size_t);
char *  index       (const char *, int);
char *  rindex      (const char *, int);

int     strcasecmp  (const char *, const char *);
int     strncasecmp (const char *, const char *, size_t);
```

The following function prototype is available through **string.h** but can be suppressed through the **_ISO_C_SOURCE** feature test macro:

```
int        strerror_r      (int, char *, size_t);
```

The following VxWorks-specific function prototypes are also available through **string.h**, but only if the **_POSIX_C_SOURCE**, and **_POSIX_AEP_RT_CONTROLLER_C_SOURCE**, and **_XOPEN_SOURCE** and **_ISO_C_SOURCE** feature test macros are **NOT** defined by the including source file:

```
void    binvert     (void *, size_t);
void    bswap       (void *, void *, size_t);
void    uswab       (const void *, void *, size_t);
void    bcopyBytes  (const void *, void *, size_t);
void    bcopyWords  (const void *, void *, size_t);
void    bcopyLongs  (const void *, void *, size_t);
void    bcopyQuads  (const void *, void *, size_t);
void    bfill       (void *, size_t, int);
```

```
void    bfillBytes  (void *, size_t, int);

void *  memmem      (const void * haystack, size_t haystackLen,
                     const void * needle, size_t needleLen);
void *  memimem     (const void * haystack, size_t haystackLen,
                     const void * needle, size_t needleLen);

size_t  strlcpy     (char *, const char *, size_t);
size_t  strlcat     (char *, const char *, size_t);
```

Summary of Changes to strings.h and string.h on page 16

3.1. Summary of Changes to strings.h and string.h

- The following function prototypes are available through both **string.h** and **strings.h**, but can be suppressed through the **_ISO_C_SOURCE** feature test macro.:

```
int      bcmp        (const void *, const void *, size_t);
void    bcopy       (const void *, void *, size_t);
void    bzero       (void *, size_t);
char *  index       (const char *, int);
char *  rindex      (const char *, int);
int     strcasecmp  (const char *, const char *);
int     strncasecmp (const char *, const char *, size_t);
```

- The following function prototypes are now available through string.h and strings.h, but only if the **_POSIX_C_SOURCE**, and **_POSIX_AEP_RT_CONTROLLER_C_SOURCE**, and **_XOPEN_SOURCE**, and **_ISO_C_SOURCE** feature test macros are **NOT** defined by the including source file :

```
void    binvert     (void *, size_t);
void    bswap       (void *, void *, size_t);
void    uswab       (const void *, void *, size_t);
void    bcopyBytes  (const void *, void *, size_t);
void    bcopyWords  (const void *, void *, size_t);
void    bcopyLongs  (const void *, void *, size_t);
void    bcopyQuads  (const void *, void *, size_t);
void    bfill       (void *, size_t, int);
void    bfillBytes  (void *, size_t, int);

void *  memmem      (const void * haystack, size_t haystackLen,
                     const void * needle, size_t needleLen);
void *  memimem     (const void * haystack, size_t haystackLen,
                     const void * needle, size_t needleLen);
```

- The various _s( ) suffixed function prototypes as per “Annex K: Bounds-checking interfaces” are now available through **string.h**

Parent topic: String Handling Interfaces: strings.h and string.h on page 12

4. Multibyte/wide Character Interfaces

Previously, the following multibyte function prototypes were provided in **stdlib.h**:

WNDRVR

```
int          mblen        (const char *__s, size_t __n);
size_t       mbstowcs     (wchar_t *__wcs, const char *__s, size_t __n);
int          mbtowc       (wchar_t *__pwc, const char *__s, size_t __n);
size_t       wcstombs     (char *__s, const wchar_t *__wcs, size_t __n);
int          wctomb       (char *__s, wchar_t __wchar);
```

However, these functions were **essentially** unimplemented. They merely performed some parameter validation and then returned OK (1) if the validation succeeded without performing any other operations.

As a result of the **C11** standard library upgrade, the following function prototypes are still provided in **stdlib.h**:

```
int         mblen         (const char *, size_t);
size_t      mbstowcs      (wchar_t *_Restrict, const char *_Restrict,
                           size_t);
int         mbtowc        (wchar_t *_Restrict, const char *_Restrict,
                           size_t);
size_t      wcstombs      (char *_Restrict, const wchar_t *_Restrict,
                           size_t);
int         wctomb        (char *, wchar_t);
```

The implementations of these functions conform to the **C11** specification. Note that the application of the "restrict" keyword now applies to several parameters (see bolded entries).

5. Formatted Input/Output Interfaces, e.g. printf(), scanf(), etc.

This section provides details about the interfaces listed below:

- The %s Conversion Specifier on page 17
- Return Value from scanf(), fscanf() and sscanf() Family of Functions on page 18
- Input Formatting of Floating Point Numbers on page 18
- Invalid Conversion Specifier on page 19
- Output Formatting of Floating Point Numbers on page 19

The %s Conversion Specifier on page 17 Previously, providing a NULL pointer to the %s specifier resulted in the string "(null)" being inserted into the output stream, e.g.:
Return Value from scanf(), fscanf() and sscanf() Family of Functions on page 18 Previously, in case of a matching failure before any successful conversion, the function returned **EOF**. For example:
Input Formatting of Floating Point Numbers on page 18 Previously, the scanf( ), fscanf( ) and sscanf( ) family of functions supported input parsing of floating point numbers from a string such as "**0.1234eearly**". For example,
Invalid Conversion Specifier on page 19 Previously, an invalid conversion specification in the printf( ), fprintf( ) and sprintf( ) family of functions resulted in omitting the **%** character, e.g.
Output Formatting of Floating Point Numbers on page 19 Previously, when formatting a floating point number for output using the printf( ), fprintf( ) and sprintf( ) family of functions, all decimal digits up to the specified precision will appear in the formatted string. For example:
Vector and Fixed-Point Conversion Specifiers on page 19 Previously, the formatted I/O interfaces supported PowerPC AltiVec and Signal Processing Engine (SPE) related conversion specifiers. The AltiVec related conversion specifiers (vector) were available when CPU=PPCE6500 while the SPE related conversion specifiers (fixed-point) were available when CPU=PPCE500V2:

5.1. The %s Conversion Specifier

Previously, providing a NULL pointer to the %s specifier resulted in the string "(null)" being inserted into the output stream, e.g.:

```
-> printf ("%s\n", 0)
(null)
value = 7 = 0x7
```

As a result of the **C11** standard library upgrade, providing a NULL pointer to the % specifier is interpreted as a "bad" argument, e.g.:

```
-> printf ("%s\n", 0)
printf_s: bad %s argument
value = 39 = 0xffffffd9 = trgNone + 9fc99081
```

More specifically, the runtime constraint handler as defined in "*Annex K: Bounds-checking interfaces*" of the **C11** specification is invoked to handle the "bad argument" condition. By default, the runtime constraint handler performs an fputs( ) of the error string, "**printf_s: bad %s argument**" in this case, and returns **ERANGE**.

The function set_constraint_handler_s( ) can be used to install an alternate runtime constraint handler.

Parent topic: Formatted Input/Output Interfaces, e.g. printf(), scanf(), etc. on page 17

5.2. Return Value from scanf(), fscanf() and sscanf() Family of Functions

Previously, in case of a matching failure before any successful conversion, the function returned **EOF**. For example:

```
sscanf ("-689", "%*d", &assignedVal);
```

In the above example, the return value of sscanf( ) was **EOF**.

As a result of the **C11** standard library upgrade, the return value in the above example will be 0. This return value corresponds to the behavior defined in the *ISO/IEC 9899:2011* specification which states:

"The sscanf function returns the value of the macro EOF if an input failure occurs before the first conversion (if any) has completed. Otherwise, the sscanf function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure."

Parent topic: Formatted Input/Output Interfaces, e.g. printf(), scanf(), etc. on page 17

5.3. Input Formatting of Floating Point Numbers

Previously, the scanf( ), fscanf( ) and sscanf( ) family of functions supported input parsing of floating point numbers from a string such as "**0.1234eearly**". For example,

```
sscanf ("0.1234eearly", "%f%s", &variable, strptr);
```

In the above example, sscanf( ) returned 2, the 'variable' would be set to *0.1234* , and '**strptr**' would contain *"eearly"*.

As a result of the **C11** standard library upgrade, the return value in the above example will be 0, while *'variable'* and *'strptr'* will be unmodified. The reason for this behaviour is due to the following footnote from the *ISO/IEC 9899:2011* specification:

"fscanf pushes back at most one input character onto the input stream. Therefore, some sequences that are acceptable to strtod, strtol, etc., are unacceptable to fscanf."

The sccanf( ) function interprets *"0.1234e"* as the start of a floating point number with an exponent. However, when the second 'e' is scanned, the implementation determines that the exponent is missing but given a push back of at most one character, sscanf( ) can only pushback the second "e" character, which results in "**0.1234e**". The string "**0.1234e"** does not represent a valid floating point number, and thus a matching failure occurs (which results in a return value of 0).

WNDRVR

Parent topic: Formatted Input/Output Interfaces, e.g. printf(), scanf(), etc. on page 17

5.4. Invalid Conversion Specifier

Previously, an invalid conversion specification in the printf( ), fprintf( ) and sprintf( ) family of functions resulted in omitting the **%** character, e.g.

```
-> printf ("%Hd\n", 34)
Hd
value = 3 = 0x3
```

As a result of the **C11** standard library upgrade, the **%** character will be included in the output, e.g.

```
-> printf ("%Hd\n", 34);
%Hd
value = 4 = 0x4
```

Note: The *ISO/IEC 9899:2011* specification indicates that: *"If a conversion specification is invalid, the behavior is undefined"*.

Parent topic: Formatted Input/Output Interfaces, e.g. printf(), scanf(), etc. on page 17

5.5. Output Formatting of Floating Point Numbers

Previously, when formatting a floating point number for output using the printf( ), fprintf( ) and sprintf( ) family of functions, all decimal digits up to the specified precision will appear in the formatted string. For example:

```
double dbl = 7.118807338895066760773380183537780600007e-184;
(void) printf("%.39e\n",dbl);
```

In the above example, the output string was "**7.118807338895066760773380183537780600007e-184**".

As a result of the **C11** standard library upgrade, only the significant decimal digits in the floating point number will appear in the formatted string. The IEEE 754 **single-precision** format has a 24-bit significand which provides 6 to 9 significant decimal digits of precision while the IEEE 754 **double-precision** format has a 53-bit significand which provides 15 to 17 significant decimal digits of precision. The remaining decimals up to the specified precision will be zero. Thus in the above example, the output string will now be "**7.118807338895067000000000000000000000000e-184**".

Parent topic: Formatted Input/Output Interfaces, e.g. printf(), scanf(), etc. on page 17

5.6. Vector and Fixed-Point Conversion Specifiers

Previously, the formatted I/O interfaces supported PowerPC AltiVec and Signal Processing Engine (SPE) related conversion specifiers. The AltiVec related conversion specifiers (vector) were available when CPU=PPCE6500 while the SPE related conversion specifiers (fixed-point) were available when CPU=PPCE500V2:

- the 'v' modifier (vector) to the standard conversion specifiers, along with the 'r' and 'R' (signed and unsigned fixed-point) conversion specifiers was supported by the printf( ), fprintf( ), and sprintf( ) family of functions
- the 'v' modifier (vector) to the standard conversion specifiers was supported by the scanf( ), fscanf( ), and sscanf( ) family of functions

For example, the following source code fragment:

WNDRVR

```
__vector unsigned char s = (__vector unsigned char)

    {'0','1','2','3','4','5','6','7','8','9','A','B','C','D','E','F'};



__vector float f = (__vector float) {-3.1415926, 3.1415926, 9.8, 0.000};



printf ("vector char s  = %,vc\n", s);

printf ("vector float f = %,5.6ve\n", f);
```

… resulted in the following formatted output:

```
vector char s  = 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

vector float f = -3.141593e+00,3.141593e+00,9.800000e+00,0.000000e+00
```

As a result of the **C11** standard library upgrade, the 'v' modifier (vector) to the standard conversion specifiers, along with the 'r' and 'R' (signed and unsigned fixed-point) conversion specifiers are no longer supported. For example, the above source code fragment will result in the following formatted output:

```
vector char s  = %,vc

vector float f = %,5.6ve
```

Parent topic: Formatted Input/Output Interfaces, e.g. printf(), scanf(), etc. on page 17

6. Buffered I/O: General

Previously, performing a buffered I/O operation, e.g. fclose( ), fread( ), clearerr( ), fflush( ), etc. on an already closed file stream usually resulted in an **EOF** return value. The **C11** specification considers operations on an already closed stream as "undefined behavior". As a result of the **C11** standard library upgrade, performing a buffered I/O operation on an already closed file stream will result in undefined behavior which could include memory corruption and/or an exception.

Previously, performing an fdopen( ) operation on an invalid file descriptor, i.e. a file descriptor that was out of range or did not represent an opened file, would result in a NULL return value. As a result of the **C11** standard library upgrade, performing an fdopen( ) operation on an invalid file descriptor will result in undefined behavior which could include memory corruption and/or an exception.

The following is the signature of the fwrite( ) interface:

```
size_t fwrite     (const void *restrict ptr, size_t size, size_t nitems,
                    FILE *restrict stream);
```

Previously, performing an fwrite( ) operation with the 'size' parameter set to 0 resulted in the function returning '**nitems**'. The *POSIX.1-2008* specification for the fwrite( ) interface indicates that "*If size or nitems is 0, fwrite() shall return 0*", thus the traditional behavior was non-conforming. As a result of the **C11** standard library upgrade, performing an fwrite( ) operation with the 'size' parameter set to 0 results in 0 being returned.

Previously, **fpos_t** was typedef'ed to a 'long'. As a result of the **C11** standard library upgrade, **fpos_t** is typedef'ed to a structure.

Previously, the number of streams (FILE *) that could be simultaneously available was only limited by the amount of memory in the kernel system heap. However, there are a fixed number of file descriptors available in the kernel as determined by the NUM_FILES configuration parameter for the **INCLUDE_IO_BASIC** component. As a result of the C11 standard library upgrade, the number of streams that can be simultaneously available is determined by the **FOPEN_MAX_KERNEL VSB** configuration parameter:

```
config FOPEN_MAX_KERNEL
    int "Max FILE pointers in kernel"
    default 50
    help
        This option sets the maximum number of open FILE pointers in the kernel.
```

The fixed number of file descriptors available is still determined by the **NUM_FILES** configuration parameter under the **INCLUDE_IO_BASIC** component.

As a result of the **C11** standard library upgrade, the output of the stdioShow( ) function no longer displays the "**Owning Task**", "**Line Buffer Base**", "**Line Buffer Size**", "**stat.st_blksize**", "**lseek Offset**", and "**Flags**" fields. For example, the following is the output of stdioShow( ) as a result of the **C11** standard library upgrade:

```
-> stdioShow 0x88901b28
File Descriptor     : 0
Current Position    : 0
Read Space Left     : 0
Write Space Left    : 0
Buffer Base         : 0
Buffer Size         : 0
Ungetc Buffer Base  : 0
Ungetc Buffer Size  : 0
Mode                : 0x3
value = 0 = 0x0
```

7. Buffered I/O: 'stdin', 'stdout', 'stderr'

Previously, a FILE structure was allocated for '**stdin**', '**stdout**', and '**stderr**' for each kernel task. Performing fread( ) operations against '**stdin**' results in read( ) operations against file descriptors **0 (STD_IN)**. Similarly, performing fwrite( ) operations against '**stdout**' or '**stderr**' result in **write()** operations against file descriptor **1 (STD_OUT)** or **2 (STD_OUT)**, respectively.

The mapping of a task's **STD_IN, STD_OUT**, and **STD_ERR** file descriptor to a "real" file descriptor, i.e. a file descriptor obtained by performing an open( ) on an I/O device, involves two levels of redirection. First, there is a global assignment of these file descriptors which new tasks use by default. The global assignment of the three standard file descriptors is controlled by the routines ioGlobalStdSet( )and ioGlobalStdGet( ). Second, individual tasks may override the global assignment of these file descriptors with their own assignments that apply only to that task. The assignment of task-specific standard file descriptors is controlled by the routines ioTaskStdSet( ) and ioTaskStdGet( ).

As a result of the **C11** standard library upgrade, only a single (aka global) FILE structure is allocated for '**stdin**', '**stdout**', and '**stderr**'. These three FILE structures are "shared" by all kernel tasks. Performing fread( )/fwrite( ) operations against '**stdin**', '**stdout**', and '**stderr**' still results in read( )/write( ) operations against file descriptors **0 (STD_IN)**, **1 (STD_OUT)**, and **2 (STD_ERR)**. However, the following are changes to the behavior and functionality:

- Read/write operations from/to '**stdin**', '**stdout**', and '**stderr**' are always unbuffered. There is no mechanism or API to allow a particular task's '**stdout**', for example, to be performed with buffering.
- The following interface to "*return the standard input/output/error FILE of the current task*" is no longer provided:

```
FILE * stdioFp (int stdFd);
```

- The freopen( ) interface is typically used to attach the already open streams '**stdin**', '**stdout**', and '**stderr**' to another file. If a kernel task performs an freopen( ) against '**stdin**', '**stdout**', or '**stderr**', the reassignment to another file affects all kernel tasks performing operations against '**stdin**', '**stdout**', or '**stderr**'.

> Note: Read/write operations from/to '**stdin**', '**stdout**', and '**stderr**' will be buffered if the new file is not a TTY device.

Changing of a task's **STD_IN, STD_OUT**, and **STD_ERR** file descriptor through the functions ioGlobalStdSet( ) and ioTaskStdSet( ) is still supported. For example, a task could use ioTaskStdSet( ) to change **STD_OUT** to a socket file descriptor. However, the fwrite( ) operations to '**stdout**' is unbuffered (and cannot be changed).

Another change in behavior not related to the single (aka global) FILE structure that is allocated for '**stdin**', '**stdout**', and '**stderr**' relates to performing an fclose( ) or freopen( ) operation on the standard files. Previously, performing an fclose( ) or freopen( ) on '**stdin**', '**stdout**', and '**stderr**' did not result in a corresponding close( ) on **STD_IN, STD_OUT**, and **STD_ERR**, respectively.

As a result of the **C11** standard library upgrade, performing an fclose( ) or freopen( ) on '**stdin**', 'stdout( )', and '**stderr**' results in a corresponding close( ) on **STD_IN, STD_OUT**, and **STD_ERR,** respectively.

8. General Utility Interfaces: stdlib.h

Previously, the following function prototypes were provided if the **_EXTENSION_WRS** macro was defined.

```
void     div_r      (int numer, int denom, div_t * divStructPtr);
void     ldiv_r     (long numer, long denom, ldiv_t * divStructPtr);
```

The **_EXTENSION_WRS** macro is defined to 1 in **vxAnsi.h**, i.e. the above function prototypes were always visible.

As a result of the **C11** standard library upgrade, the div_r( ) and ldiv_r( ) functions are no longer provided. The div( ) and ldiv( ) interfaces are re-entrant, thus existing usage of div_r( ) and ldiv_r( ) can easily be updated to use div( ) and ldiv( ).

> Note: Any compatibility issues related to environment variable interfaces are covered in Environment Variables on page 9.

9. Mathematical Interfaces: math.h

Previously, the following function prototypes were provided through **math.h:**

```
double   scalb       (double x, double exp);
int      finite      (double x);
double   drem        (double x, double y);
```

As a result of the **C11** standard library upgrade, these functions are no longer provided. The functionality of these functions is provided by the following (introduced with C99):

```
double   scalebln      (double x, long int exp);
int      isfinite      (real-floating x);
double   remainder     (double x, double y);
```

> Note: The isfinite( ) API is implemented as a macro.

WNDRVR

10. Time zone: TIMEZONE and TZ Environment Variables

Previously, the format of the *TIMEZONE* environment variable was as follows:

```
<name of zone>::<time in minutes from UTC>:<daylight start>:<daylight end>
```

For example: "**UTC::0:040102:100102**"

The default time zone setting, i.e. when the TIMEZONE environment variable was not set, was *"UTC::::"*

As a result of the **C11** standard library upgrade, the format of the *TIMEZONE* environment variable is as follows:

```
:EST:EDT:-0500:031502-0:110802-0
```

The following is a decoding of the above example:

- **EST** - Designation for the standard time zone
- **EDT** - Designation for the alternative time zone, e.g. daylight savings time zone
- **-0500** - EST is 5 hours and 00 minutes behind UTC
- **031502-0** - EDT begins on the first Sunday before (-0) 15 March (0315) at 02:00 (02)
- **110802-0** - EDT ends on the first Sunday before (-0) 8 November (1108) at 02:00 (02)

Also, the *POSIX.1-2008 TZ* environment variable is supported. If both the *TZ* and *TIMEZONE* environment variables have been set, then the value of the *TZ* environment variable shall take precedence.

The default time zone setting, i.e. when both the TZ and TIMEZONE environment variables are not set, is :

```
:UTC:UTC:0000
```

The **POSIX.1-2008** specified tzset( ) interface is provided to use the value of the *TZ* or *TIMEZONE* environment variables for usage by the ctime( ) , localtime( ), mktime( ), and strftime( )interfaces.

Note: The **TIMEZONE_ENV_VAR** parameter of the **INCLUDE_DEFAULT_TIMEZONE** component can be used to change the default time zone setting when both the *TZ* and *TIMEZONE* environment variables are not set.

11. Input and Output Interfaces: stdio.h

Previously, the following interfaces were provided through **stdio.h:**

```
void     setbuf         (FILE *stream, char *buf);
void     setbuffer      (FILE *stream, char *buf, size_t size);
void     setlinebuf     (FILE *stream);
int      setvbuf        (FILE *stream, char *buf, int mode, size_t size);
```

As a result of the **C11** standard library upgrade, only the following interfaces are available through **stdio.h**:

```
void    setbuf       (FILE *stream, char *buf);
int     setvbuf      (FILE *stream, char *buf, int mode, size_t size);
```

The setbuffer( ) and setlinebuf( ) functions were just special cases of setvbuf( ), and invocations of these functions can be replaced in applications as follows:

- setbuffer ( )

```
setvbuf  (stream, buf, _IOFBF, size);
        or
setvbuf  (stream, NULL, _IONBF), (size_t) 0);
```

- setlinebuf( )

```
setvbuf (stream, (char *) NULL, _IOLBF, (size_t) 0);
```

As a consequence of the elimination of setbuffer( ), the following **VIP** component has also been eliminated:

```
Component INCLUDE_ANSI_STDIO_EXTRA {
        NAME            ANSI stdio extensions
        SYNOPSIS        This component includes the Wind River routines: \
                        getw(), putw(), and setbuffer().
        LINK_SYMS       getw putw setbuffer
}
```

The **C11** specified getw( ) and putw( ) functions are provided by the **INCLUDE_ANSI_STDIO** component.

Previously, the following interfaces were also provided through **stdio.h**:

```
int      oprintf       (OPRINTF_OUTPUT_FUNCPTR, _Vx_usr_arg_t, const char *,
                        ...);
int      voprintf      (OPRINTF_OUTPUT_FUNCPTR, _Vx_usr_arg_t, const char *,
                        va_list);
int      fdprintf      (int, const char *  , ...);
int      vfdprintf     (int, const char *  , va_list);
int      printErr      (const char *  , ...);
int      kprintf       (const char *, ...);
ssize_t  kputs         (char *);
FILE *   stdioFp       (int);
STATUS   stdioShow     (FILE *, int);
```

As a result of the **C11** standard library upgrade, the following interfaces have moved to **fioLib.h**:

```
int      oprintf       (OPRINTF_OUTPUT_FUNCPTR, _Vx_usr_arg_t, const char *,
                        ...);
int      voprintf      (OPRINTF_OUTPUT_FUNCPTR, _Vx_usr_arg_t, const char *,
                        va_list);
int      fdprintf      (int, const char *  , ...);
int      vfdprintf     (int, const char *  , va_list);
int      printErr      (const char *  , ...);
int      kprintf       (const char *, ...);
ssize_t  kputs         (char *);

STATUS   stdioShow     (FILE *, int);
```

As mentioned in section Buffered I/O: 'stdin', 'stdout', 'stderr' on page 21, the stdioFp( ) interface is no longer provided.

The following function prototypes are also available through **stdio.h** but can be suppressed through the **_ISO_C_SOURCE** feature test macro:

```
int      getc_unlocked         (FILE *);
int      getchar_unlocked      (void);
int      putc_unlocked         (int, FILE *);
int      putchar_unlocked      (int);
int      fseeko                (FILE *, off_t, int);
off_t    ftello                (FILE *str);
void     flockfile             (FILE *file);
void     funlockfile           (FILE *file);
int      ftrylockfile          (FILE *file);
```

12. Debugging Thread-Local Storage (TLS) Variables

Switching to the Clang compiler for ARM 32-bit and Intel 32-bit & 64-bit, along with switching to a non-Wind River customized GNU compiler for PowerPC necessitated adopting the "Linux TLS ABI" in the kernel environment for these architectures. In previous releases, the ARM 64-bit architecture already adopted the Linux TLS ABI in the kernel environment, and all the remaining architectures adopted the Linux TLS ABI in the RTP environment. In this release, all architectures have adopted the Linux TLS ABI in both the kernel and RTP environments.

Due to "missing data" in the **DWARF** information for *TLS* variables in a relocatable object module, the debugger is not able to examine *TLS* variables defined in a DKM. Specifically, the @DW_AT_location attribute for *TLS* variables in a DKM are always zero. Thus it is not possible for the debugger to differentiate between multiple *TLS* variables in an individual DKM. An attempt to examine a *TLS* variable defined in a DKM will result in the message: "*Cannot get symbol location information. Examining thread-local storage variables in a DKM is not supported*".

13. POSIX.1-2008 Input and Output Interfaces

Previously, the following interfaces were provided in **unistd.h**:

```
ssize_t  read      (int fd, char * buffer, size_t maxbytes);
ssize_t  write     (int fd, const char * buffer, size_t nbytes);
```

As a result of the **C11** standard library upgrade, the signatures have changed to conform to **POSIX.1-2008**:

```
ssize_t  read       (int fd, void * buffer, size_t maxbytes);
ssize_t  write      (int fd, const void * buffer, size_t nbytes);
```

Previously, the following interface was provided in **ioLib.h**:

```
ssize_t  open   (const char * path, int flags, int mode);
```

As a result of the **C11** standard library upgrade, the signatures have changed to conform to **POSIX.1-2008**:

```
ssize_t  open   (const char * path, int flags, ...);
```

14. Non-Local Jumps: setjmp.h

The following function prototypes are available through **setjmp.h** and can be suppressed through the **_ISO_C_SOURCE** feature test macro:

```
int      sigsetjmp      (sigjmp_buf,int);
void     siglongjmp     (sigjmp_buf, int);
```

15. Localization

In VxWorks, **C** standard localization is the default setting. However, you can use the **C11** standard library for many European or American locales in your applications. For the complete list, see the *installDir* **/vxworks-7/pkgs_v2/os/lang_lib/libc/libc_std/locales** directory. Each file in this directory defines a European or American locale (for example, the **en_CA** file is used for the English language locale in Canada). The directory also includes a **readme.txt** file that includes code examples.

Procedure

1. On host machine, concatenate the locales you want to use in a single file.

   To use all the locales:

   ```
   $ cat POSIX *_* > /mydir/locales.txt
   ```

2. (Optional) If your **locales.txt** file has DOS newline characters (**\r\n**), replace them with the Unix newline character (**\n**).

   ```
   $ dos2unix /mydir/locales.txt
   ```

3. Copy the **locales.txt** file to your target storage (for example, **/romfs/locales.txt**).
4. In your RTP on the target, set the environment variable *LOCFILE* to point to the **locales.txt** file.

   If **locales.txt** is located at **/romfs/locales.txt**:

   ```
   putenv("LOCFILE=/romfs/locales.txt")
   ```

   -or-

   ```
   setenv("LOCFILE", "/romfs/locales.txt", 1)  /* the 3rd argument 1 means overwriting if environm
   ent variable LOCFILE already exists  */
   ```

5. Set the desired locale for your RTP using the setlocale command.

   ```
   $ setlocale(LC_ALL, "de_AT")
   ```

6. KERNEL ENVIRONMENT: C++14/C++17 COMPATIBILITY

1. Kernel Environment: C++14/C++17 Compatibility

Previously, the kernel environment provided libraries based on the original **C++** standard (amended by the technical corrigendum approved in 2003), ISO/IEC 14882:2003.

In this release, the kernel environment provides libraries conforming to the **C++ 17** standard, ISO/IEC 14882:2017.

Application code will continue to be compiled as C++14 by default. To use C++17 in an application or layer, add the following compiler option to the build tool flags:

```
-std=c++17
```

Namespace for C Routines on page 27 The following is an extract from the **C++14** standard:
Wrappers Supporting Interfaces using FUNCPTR on page 29 As a result of the **C++14/C++17** standard library upgrade, the series of **C++** inline wrappers that support VxWorks native interfaces using the **FUNCPTR** type definition have been removed. These wrappers were introduced in VxWorks 6.x to maintain backwards compatibility for **C++** applications that used VxWorks native interfaces that were changed to use fully-qualified function pointers instead of **FUNCPTR**, **VOIDFUNCPTR**, **DBLFUNCPTR**, and **FLTFUNCPTR**.
C++ Exception Handling on page 30**C++** exception handling is provided by the **INCLUDE_CPLUS_LANG** component. Previously, exception handling could be disabled for user code by the following steps:
Kernel C++17 Compatibility on page 30 Users wishing to migrate to C++17 should be aware of a few ways in which C++17 is backward incompatible with C++14.

1.1. Namespace for C Routines

The following is an extract from the **C++14** standard:

```
2 Every C header, each of which has a name of the form name.h, behaves as if each name placed in t
he standard library namespace by the corresponding cname header is placed within the global namesp
ace scope. It is unspecified whether these names are first declared or defined within namespace sc
ope (3.3.6) of the namespace std and are then injected into the global namespace scope by explici
t using-declarations (7.3.3).
3 [ Example: The header <cstdlib> assuredly provides its declarations and definitions within the n
amespace std. It may also provide these names within the global namespace. The header <stdlib.h> a
ssuredly provides the same declarations and definitions within the global namespace, much as in th
e C Standard. It may also provide these names within the namespace std. —end example ]
```

Using the printf( ) function as an example, according to the above extract, the following constructs must be supported by a conforming implementation:

```
a)      #include <stdio.h>

void foo (void)
    {
    ::printf ("Hello World\n");
    }

b)      #include <cstdio>
```

```
void foo (void)
    {
    std::printf ("Hello world\n");
    }
```

Also, according to the above extract, the following constructs may or may not be supported by a conforming implementation:

```
a)      #include <stdio.h>

void foo (void)
    {
    std::printf ("Hello world\n");
    }

b)      #include <cstdio>

void foo (void)
    {
    ::printf ("Hello world\n");
    }
```

As a result of the **C11/C++14/C++17** standard library upgrade, **C++** sources that include **C** header files, e.g. **stdio.h**, have access to the **C** interfaces only in the global namespace. However, including the **C++** wrapper header, e.g. '**cstdio**', results in the **C** interfaces being placed in both the global and "std" namespaces. In other words, the following constructs are supported:

```
a)      #include <stdio.h>

void foo (void)
    {
    ::printf ("Hello World\n");
    }

b)      #include <cstdio>

void foo (void)
    {
    std::printf ("Hello world\n");
    }

c)      #include <cstdio>

void foo (void)
    {
    ::printf ("Hello world\n");
    }
```

This results in a backwards compatibility issue in that existing **C++** code that includes **stdio.h**, for example, and then attempts to reference printf( ) in the "std" namespace **(std::printf())**, no longer works.

The rationale for selecting this approach is that: a) it is consistent with Linux, thus making it easier to port existing Linux **C++** applications to VxWorks, and b) the historical practice of including the **C++** wrapper headers with the expectation that global names space will be supported.

Parent topic: Kernel Environment: C++14/C++17 Compatibility on page 27

1.2. Wrappers Supporting Interfaces using FUNCPTR

As a result of the **C++14/C++17** standard library upgrade, the series of **C++** inline wrappers that support VxWorks native interfaces using the **FUNCPTR** type definition have been removed. These wrappers were introduced in VxWorks 6.x to maintain backwards compatibility for **C++** applications that used VxWorks native interfaces that were changed to use fully-qualified function pointers instead of **FUNCPTR**, **VOIDFUNCPTR**, **DBLFUNCPTR**, and **FLTFUNCPTR**.

The following are the **C++** inline wrappers that have been deleted:

- **tyLib.h**

```
inline STATUS tyDevInit          (FAST TY_DEV_ID, size_t, size_t, FUNCPTR)
inline STATUS tyXoffHookSet      (TY_DEV_ID, FUNCPTR, _Vx_usr_arg_t)
```

- **dllLib.h**

```
inline DL_NODE * dllEach (DL_LIST *, FUNCPTR, _Vx_usr_arg_t)
```

- **excLib.h**

```
inline void excHookAdd (FUNCPTR)
```

- **hashLib.h**

```
inline HASH_NODE * hashTblEach(HASH_ID,FUNCPTR,_Vx_usr_arg_t)
```

- **logLib.h**

```
inline int logMsg (const char *, _Vx_usr_arg_t, _Vx_usr_arg_t,_Vx_usr_arg_t,
                   _Vx_usr_arg_t,_Vx_usr_arg_t, _Vx_usr_arg_t)
```

- **rebootLib.h**

```
inline STATUS rebootHookAdd (FUNCPTR)
```

- **sllLib.h**

```
inline SL_NODE * sllEach (SL_LIST *, FUNCPTR ,_Vx_usr_arg_t)
```

- **symLib.h**

```
inline SYMBOL_ID symEach (SYMTAB_ID, FUNCPTR, _Vx_usr_arg_t)
```

- **dosFsLib.h**

```
inline STATUS dosFsVolFormat    (const char *, int, FUNCPTR)
inline STATUS dosFsVolFormatFd  (int, int, FUNCPTR)
```

- **moduleLib.h**

```
inline MODULE_ID      moduleEach         (FUNCPTR, _Vx_usr_arg_t)
inline SEGMENT_ID     moduleSegEach      (MODULE_ID, FUNCPTR, _Vx_usr_arg_t)
```

- **stdio.h**

```
inline int oprintf      (FUNCPTR, _Vx_usr_arg_t, const char *, ...)
inline int voprintf     (FUNCPTR, _Vx_usr_arg_t, const char *, va_list)
```

- **fioLib.h**

```
inline int0 fioFormatV        (const char *,va_list vaList, FUNCPTR,
                               _Vx_usr_arg_t)
inline int  fioScanV          (const char *, FUNCPTR, void *, int *, va_list)
```

Parent topic: Kernel Environment: C++14/C++17 Compatibility on page 27

1.3. C++ Exception Handling

C++ exception handling is provided by the **INCLUDE_CPLUS_LANG** component. Previously, exception handling could be disabled for user code by the following steps:

Procedure

1. Define the environment variable *CXX_STL_VARIANT=abridged* before creating the VxWorks Image Project (VIP). This ensures that the correct (exception-disabled) libraries are linked against.
2. Add **-mc++-abr** for GNU or **-Xc++-abr** for Diab to the compiler options.

Results

As a result of the **C++14/C++17** standard library upgrade, **C++** exception handling can no longer be disabled.

Parent topic: Kernel Environment: C++14/C++17 Compatibility on page 27

1.4. Kernel C++17 Compatibility

Users wishing to migrate to C++17 should be aware of a few ways in which C++17 is backward incompatible with C++14.

For a full list of changes see:

https://isocpp.org/files/papers/p0636r0.html

In particular dynamic exception specifications are no longer supported. Since C++11 compile-time checked "noexcept" specifications have been preferred to dynamically checked exception "throw(A, B, ...)" specifications. In C++17, the throw(A, B, ...) syntax is no longer supported. However **throw()** with no arguments is still allowed but is now an alias for "noexcept". Support for **throw()** will be removed in C++20.

The VxWorks implementation of C++17 has the following limitation:

- Parallel algorithms (for example **par_reduce**) are not yet supported.

Parent topic: Kernel Environment: C++14/C++17 Compatibility on page 27

WNDRVR

7. RTP ENVIRONMENT: C11 COMPATIBILITY

1. Environment Variables

Previously the following function prototypes were available through **stdlib.h**:

```
char *        getenv        (const char *__name);
errno_t       getenv_s      (size_t *_Restrict, char *_Restrict, rsize_t,
                             const char *_Restrict);
int           putenv        (char *);
int           setenv        (const char *, const char *, int);
int           unsetenv      (const char *);
```

As a result of the **C11** standard library upgrade, the following environment variable handling function prototypes are available through **stdlib.h** :

```
errno_t       getenv_s      (size_t *_Restrict, char *_Restrict, rsize_t,
                             const char *_Restrict);
char *        getenv        (const char *);
```

The following function prototype is also available through **stdlib.h** but can be suppressed through the **_POSIX_C_SOURCE** feature test macro:

```
int putenv (char *string);
```

The following function prototypes are also available through **stdlib.h** but can be suppressed through the **_ISO_C_SOURCE** feature test macro:

```
int           setenv        (const char *, const char *, int);
int           unsetenv      (const char *);
```

2. String Handling Interfaces: strings.h and string.h

Previously, the following interfaces were provided in **strings.h** :

```
int      bcmp        (const void *, const void *, size_t);
void    bcopy       (const void *, void *, size_t);
void    bzero       (void *, size_t);
char *  index       (const char *, int);
char    rindex      (const char *, int);

int     strcasecmp  (const char *, const char *);
int     strncasecmp (const char *, const char *, size_t);
```

The following function prototypes were also available through strings.h, but only if the **_POSIX_C_SOURCE**, and **_POSIX_AEP_RT_CONTROLLER_C_SOURCE**, and **_XOPEN_SOURCE** and **_ISO_C_COURCE** feature test macros were **NOT** defined by the including source file:

WNDRVR

```
void    binvert     (void *, size_t);
void    bswap       (void *, void *, size_t);
void    uswab       (const void *, void *, size_t);
void    bcopyBytes  (const void *, void *, size_t);
void    bcopyWords  (const void *, void *, size_t);
void    bcopyLongs  (const void *, void *, size_t);
void    bcopyQuads  (const void *, void *, size_t);
void    bfill       (void *, size_t, int);
void    bfillBytes  (void *, size_t, int);
```

Previously, the following interfaces were provided in **string.h**:

```
int    memcmp          (const void *, const void *, size_t);
void * memcpy          (void *_Restrict, const void *_Restrict, size_t);
void * memset          (void *, int, size_t);
char * strcat          (char *_Restrict, const char *_Restrict);
int    strcmp          (const char *, const char *);
char * strcpy          (char *_Restrict, const char *_Restrict);
size_t strlen          (const char *);

void * memmove         (void *, const void *, size_t);
int    strcoll         (const char *, const char *);
size_t strcspn         (const char *, const char *);
char * strerror        (int);
char * strncat         (char *_Restrict, const char *_Restrict, size_t);
int    strncmp         (const char *, const char *, size_t);
char * strncpy         (char *_Restrict, const char *_Restrict, size_t);
size_t strspn          (const char *, const char *);
char * strtok          (char *_Restrict, const char *_Restrict);
size_t strxfrm         (char *_Restrict, const char *_Restrict, size_t);

void * memmem          (const void *, size_t, const void *, size_t);
void * memimem         (const void *, size_t, const void *, size_t);

char * strtok_r        (char *, const char *, char **);
int    strerror_r      (int, char *, size_t);

void * memccpy         (void *, const void *, int, size_t);

char * strdup          (const char *);
int    strcasecmp      (const char *, const char *);
int    strncasecmp     (const char *, const char *, size_t);
char * strtok_r        (char *, const char *, char **);

char * strchr          (const char *, int);
char * strpbrk         (const char *, const char *);
char * strrchr         (const char *, int);
char * strstr          (const char *, const char *);

const void * memchr    (const void *, int, size_t);

errno_t memcpy_s       (void *_Restrict, rsize_t,
                        const void *_Restrict, rsize_t);
errno_t memmove_s      (void *, rsize_t, const void *, rsize_t);
errno_t memset_s       (void *, rsize_t, int, rsize_t);

errno_t strcpy_s       (char *_Restrict, rsize_t, const char *_Restrict);
```

```
errno_t strncpy_s      (char *_Restrict, rsize_t, const char *_Restrict,
                        rsize_t);
errno_t strcat_s       (char *_Restrict, rsize_t, const char *_Restrict);
errno_t strncat_s      (char *_Restrict, rsize_t, const char *_Restrict,
                        rsize_t);
char * strtok_s        (char *_Restrict, rsize_t *_Restrict,
                        const char *_Restrict, char **_Restrict);

errno_t strerror_s     (char *, rsize_t, errno_t);

size_t  strerrorlen_s  (errno_t);

size_t  strnlen_s      (const char *_Restrict, size_t);
```

The following function prototype was provided through string.h if the **_VXWORKS_COMPATIBILITY_MODE** macro was defined by the including source file:

```
int     strerror_r      (int, char *);
```

The following function prototypes were also available through string.h, but only if the **_POSIX_C_SOURCE**, and **_POSIX_AEP_RT_CONTROLLER_C_SOURCE**, and **_XOPEN_SOURCE** feature test macros were **NOT** defined by the including source file:

```
size_t strlcpy  (char *, const char *, size_t);
size_t strlcat  (char *, const char *, size_t);
```

As a result of the **C11** standard library upgrade, the following function prototypes are still available through **strings.h**. These prototypes can be suppressed though the **_ISO_C_SOURCE** feature test macro.

```
int     bcmp           (const void *, const void *, size_t);
void    bcopy          (const void *, void *, size_t);
void    bzero          (void *, size_t);
char *  index          (const char *, int);
char *  rindex         (const char *, int);

int     strcasecmp     (const char *, const char *);
int     strncasecmp    (const char *, const char *, size_t);
```

The following function prototypes are also available through **strings.h**, but only if the **_POSIX_C_SOURCE**, and **_POSIX_AEP_RT_CONTROLLER_C_SOURCE**, and **_XOPEN_SOURCE**, and **_ISO_C_SOURCE** feature test macros are **NOT** defined by the including source file:

```
void     binvert        (void *, size_t);
void     bswap          (void *, void *, size_t);
void     uswab          (const void *, void *, size_t);
void     bcopyBytes     (const void *, void *, size_t);
void     bcopyWords     (const void *, void *, size_t);
void     bcopyLongs     (const void *, void *, size_t);
void     bcopyQuads     (const void *, void *, size_t);
void     bfill          (void *, size_t, int);
void     bfillBytes     (void *, size_t, int);

void *   memmem         (const void *, size_t, const void *, size_t);
void *   memimem        (const void *, size_t, const void *, size_t);
```

As a result of the **C11** standard library upgrade, the following function prototypes are available through **string.h**:

```
int       memcmp         (const void *, const void *, size_t);
void *    memcpy         (void *_Restrict, const void *_Restrict, size_t);
void *    memset         (void *, int, size_t);
char *    strcat         (char *_Restrict, const char *_Restrict);
int       strcmp         (const char *, const char *);
char *    strcpy         (char *_Restrict, const char *_Restrict);
size_t    strlen         (const char *);

void *    memmove        (void *, const void *, size_t);
int       strcoll        (const char *, const char *);
size_t    strcspn        (const char *, const char *);
char *    strerror       (int);
char *    strncat        (char *_Restrict, const char *_Restrict, size_t);
int       strncmp        (const char *, const char *, size_t);
char *    strncpy        (char *_Restrict, const char *_Restrict, size_t);
size_t    strspn         (const char *, const char *);
char *    strtok         (char *_Restrict, const char *_Restrict);
size_t    strxfrm        (char *_Restrict, const char *_Restrict, size_t);
char *    strdup         (const char *);
int       strcasecmp     (const char *, const char *);
int       strncasecmp    (const char *, const char *, size_t);
char *    strtok_r       (char *, const char *, char **);
char *    strchr         (const char *, int);
char *    strpbrk        (const char *, const char *);
char *    strrchr        (const char *, int);
char *    strstr         (const char *, const char *);
void *    memchr         (const void *, int, size_t);

errno_t   memcpy_s       (void *_Restrict, rsize_t, const void *_Restrict,
                          rsize_t);
errno_t   memmove_s      (void *, rsize_t, const void *, rsize_t);
errno_t   strcpy_s       (char *_Restrict, rsize_t, const char *_Restrict);
errno_t   strncpy_s      (char *_Restrict, rsize_t, const char *_Restrict,  rsize_t);
errno_t   strcat_s       (char *_Restrict, rsize_t, const char *_Restrict);
errno_t   strncat_s      (char *_Restrict, rsize_t, const char *_Restrict,
                          rsize_t);
char *    strtok_s       (char *_Restrict, rsize_t *_Restrict,
                          const char *_Restrict, char **_Restrict);

errno_t  memset_s        (void *, rsize_t, int, rsize_t);
errno_t  strerror_s      (char *, rsize_t, errno_t);
size_t   strerrorlen_s   (errno_t);
size_t   strnlen_s       (const char *_Restrict, size_t);
int      strerror_r      (int, char *, size_t);

int      bcmp            (const void *, const void *, size_t);
void     bcopy           (const void *, void *, size_t);
void     bzero           (void *, size_t);
char *   index           (const char *, int);
char *   rindex          (const char *, int);
```

The following function prototype is available through **stdlib.h** but can be suppressed through the **_ISO_C_SOURCE** feature test macro:

```
int        strerror_r      (int, char *, size_t);
```

As a result of the **C11** standard library upgrade, the following function prototypes are still available through **string.h**. These prototypes can be suppressed though the **_ISO_C_SOURCE** feature test macro.

```
int     bcmp           (const void *, const void *, size_t);
void    bcopy          (const void *, void *, size_t);
void    bzero          (void *, size_t);
char *  index          (const char *, int);
char *  rindex         (const char *, int);

int     strcasecmp     (const char *, const char *);
int     strncasecmp    (const char *, const char *, size_t);
```

The following VxWorks-specific function prototypes are also available through **string.h**, but only if the **_POSIX_C_SOURCE**, and **_POSIX_AEP_RT_CONTROLLER_C_SOURCE**, and **_XOPEN_SOURCE** and **_ISO_C_SOURCE** feature test macros are **NOT** defined by the including source file:

```
void     binvert       (void *, size_t);
void     bswap         (void *, void *, size_t);
void     uswab         (const void *, void *, size_t);
void     bcopyBytes    (const void *, void *, size_t);
void     bcopyWords    (const void *, void *, size_t);
void     bcopyLongs    (const void *, void *, size_t);
void     bcopyQuads    (const void *, void *, size_t);
void     bfill         (void *, size_t, int);
void     bfillBytes    (void *, size_t, int);

void *   memmem        (const void *, size_t, const void *, size_t);
void *   memimem       (const void *, size_t, const void *, size_t);

size_t   strlcpy       (char *, const char *, size_t);
size_t   strlcat       (char *, const char *, size_t);
```

Summary of Changes to strings.h and string.h on page 35

2.1. Summary of Changes to strings.h and string.h

- The following function prototypes are now available through string.h and strings.h, but only if the **_POSIX_C_SOURCE**, and **_POSIX_AEP_RT_CONTROLLER_C_SOURCE**, and **_XOPEN_SOURCE**, and **_ISO_C_SOURCE** feature test macros are **NOT** defined by the including source file :

```
void    binvert      (void *, size_t);
void    bswap        (void *, void *, size_t);
void    uswab        (const void *, void *, size_t);
void    bcopyBytes   (const void *, void *, size_t);
void    bcopyWords   (const void *, void *, size_t);
void    bcopyLongs   (const void *, void *, size_t);
void    bcopyQuads   (const void *, void *, size_t);
void    bfill        (void *, size_t, int);
void    bfillBytes   (void *, size_t, int);

void *  memmem       (const void * haystack, size_t haystackLen,
                      const void * needle, size_t needleLen);
void *  memimem      (const void * haystack, size_t haystackLen,
                      const void * needle, size_t needleLen);
```


WNDRVR

- The usage of the **_VXWORKS_COMPATIBILITY_MODE** macro to obtain the following signature for strerror_r( ) is no longer supported:

```
int     strerror_r      (int, char *);
```

Parent topic: String Handling Interfaces: strings.h and string.h on page 31

3. Formatted Output Interfaces, e.g. printf(), wprintf(), etc

This section describes the output interfaces listed below:

- The %s Conversion Specifier on page 36
- The %p Conversion Specifier on page 37

The %s Conversion Specifier on page 36 Previously, with the single byte character interfaces, e.g. printf( ), providing a NULL pointer to the %s specifier resulted in the string "(null)" being inserted into the output stream, e.g.:
The %p Conversion Specifier on page 37 Previously, the "%p" format specifier resulted in no prefix characters being added to the formatted output. For example, the following printf( ) invocation:

3.1. The %s Conversion Specifier

Previously, with the single byte character interfaces, e.g. printf( ), providing a NULL pointer to the %s specifier resulted in the string "(null)" being inserted into the output stream, e.g.:

```
(void) printf ("%s\n", 0);
```

In the above example, the resultant formatted output was:

```
(null)
```

As a result of the **C11** standard library upgrade, providing a NULL pointer to the % specifier is interpreted as a "bad" argument. Thus in the above example, the resultant formatted out will be:

```
printf_s: bad %s argument
```

More specifically, the runtime constraint handler as defined in "*Annex K: Bounds-checking interfaces*" of the **C11** specification is invoked to handle the "bad argument" condition. By default, the runtime constraint handler performs an fputs( ) of the error string, "**printf_s: bad %s argument**" in this case, and returns **ERANGE**.

The function set_constraint_handler_s( ) can be used to install an alternate runtime constraint handler.

The behavior of the wide character interfaces, e.g. wprintf( ) has not changed. In other words, providing a NULL pointer to the %s specifier was, and continues to be, interpreted as a "bad" argument, e.g.:

```
(void) wprintf ("%s\n", 0);
```

In the above example, the resultant output was, and will continue to be:

```
wprintf_s: bad %s argument
```


WNDRVR

Parent topic: Formatted Output Interfaces, e.g. printf(), wprintf(), etc on page 36

3.2. The %p Conversion Specifier

Previously, the "%p" format specifier resulted in no prefix characters being added to the formatted output. For example, the following printf( ) invocation:

```
printf ("%p\n", 12345);
```

… resulted in the following formatted output:

```
3039
```

As a result of the **C11** standard library upgrade, the characters "**0x**" is prefixed . For example, the following printf( ) invocation:

```
printf ("%p\n", 12345);
```

… results in the following formatted output:

```
0x3039
```

Note: The **C11** standard indicates that the output formatting re: the "**%p"** format specifier is "*implementation defined*".

Parent topic: Formatted Output Interfaces, e.g. printf(), wprintf(), etc on page 36

4. Buffered I/O: General

Previously, the number of streams (FILE *) that could be simultaneously available in an RTP was limited to 20 (with the **_FOPMAX** macro in **yvals.h**). There are also a fixed number of file descriptors available in an RTP as determined by the **RTP_FD_NUM_MAX** configuration parameter under the **INCLUDE_RTP** component. As a result of the **C11** standard library upgrade, the number of streams that can be simultaneously available in an RTP is determined by the **FOPEN_MAX_RTP VSB** configuration parameter:

```
config FOPEN_MAX_RTP
    int "Max FILE pointers per RTP"
    depends on RTP
    default 20
    help
        This option sets the maximum number of open FILE pointers per RTP.
```

The fixed number of file descriptors available in an RTP is still determined by the **RTP_FD_NUM_MAX** configuration parameter under the **INCLUDE_RTP** component.

5. Date and Time Interfaces: time.h

Previously, the following interfaces were provided in **time.h**:

WNDRVR

```
time_t        time        (time_t *);
char *        asctime     (const struct tm *);
clock_t       clock       (void);
char *        ctime       (const time_t *);
double        difftime    (time_t, time_t);
struct tm *   gmtime      (const time_t *);
struct tm     localtime   (const time_t *);
time_t        mktime      (struct tm *);
size_t        strftime    (char *_Restrict, size_t, const char *_Restrict,
                           const struct tm *_Restrict);

errno_t       asctime_s   (char *, rsize_t, const struct tm *);
errno_t       ctime_s     (char *, rsize_t, const time_t *);
struct tm *   gmtime_s    (const time_t *_Restrict, struct tm *_Restrict);
struct tm *   localtime_s (const time_t *_Restrict, struct tm *_Restrict);

void          tzset       (void);
char  *       tzname[];
```

The following X/Open System Interface definitions were also provided in time.h when either the **_XOPEN_SOURCE** or **_POSIX_AEP_RT_CONTROLLER_C_SOURCE** feature test macros were defined by the including source code:

```
int           daylight;
long          timezone;
```

In addition, **time.h** unconditionally included **clockLib.h** and **timerLib.h**. The interfaces provided by **clockLib.h** and **timerLib.h** are **POSIX.1-2008** extensions. Thus the following interfaces were also available through the inclusion of **time.h**:

```
int     timer_create          (clockid_t clock_id,
                               struct sigevent *_Restrict evp,
                               timer_t *_Restrict ptimer);
int     timer_delete          (timer_t timerid);
int     timer_gettime         (timer_t timerid, struct itimerspec * value);
int     timer_settime         (timer_t timerid, int flags,
                               const struct itimerspec *_Restrict value,
                               struct itimerspec *_Restrict ovalue);
int     timer_getoverrun      (timer_t timerid);
int     nanosleep             (const struct timespec * rqtp,
                               struct timespec * rmtp);

int     clock_gettime         (clockid_t clock_id, struct timespec * tp);
int     clock_settime         (clockid_t clock_id, const struct timespec * tp);
int     clock_getres          (clockid_t clock_id, struct timespec * res);
int     clock_nanosleep       (clockid_t clock_id, int flags,
                               const struct timespec * rqtp,
                               struct timespec * rmtp);
```

The following VxWorks specific interfaces were also provided by **timerLib.h** (which was included by **time.h**), but only if the **_POSIX_C_SOURCE, _XOPEN_SOURCE**, and **_POSIX_AEP_RT_CONTROLLER_C_SOURCE** feature test macros were not defined:

```
int     timer_connect          (timer_t timerid, TIMER_CONNECT_FUNC routine,
                                _Vx_usr_arg_t arg);
int     timer_cancel           (timer_t timerid);
timer_t timer_open             (const char * name, int mode, clockid_t clockId,
```

```
                                 struct sigevent * evp, void * context);
STATUS  timer_close             (timer_t timerId);
STATUS  timer_unlink            (const char * name);
```

Previously, the inclusion of **time.h** also provided definitions for the following **POSIX.1-2008** extensions:

```
char * asctime_r         (const struct tm *_tm, char *_buffer);
char * ctime_r           (const time_t *_cal, char *_buffer);
struct tm * gmtime_r     (const time_t *_tod,  struct tm *_result);
struct tm * localtime_r  (const time_t *_tod, struct tm *_result);
```

Previously, the inclusion of time.h provided the following VxWorks specific variants of asctime_r( ), ctime_r( ), gmtime_r( ), and localtime_r( ) if the **_VXWORKS_COMPATIBILITY_MODE** macro was defined by the including source file:

```
int     asctime_r     (const struct tm *_tm, char *_buffer, size_t *_buflen);
char * ctime_r        (const time_t *_cal, char *_buffer, size_t *_buflen);
int     gmtime_r      (const time_t *_tod, struct tm *_result);
int     localtime_r   (const time_t *_tod, struct tm *_result);
```

The differences between the **POSIX.1-2008** and VxWorks specific definitions of the interfaces have been bolded.

As a result of the **C11** standard library upgrade, the following function prototypes are provided in **time.h** :

```
time_t        time          (time_t *);
int           timespec_get  (struct timespec *ts, int base);
char *        asctime       (const struct tm *);
clock_t       clock         (void);
char *        ctime         (const time_t *);
double        difftime      (time_t, time_t);
struct tm *   gmtime        (const time_t *);
struct tm *   localtime     (const time_t *);
time_t        mktime        (struct tm *);
size_t        strftime      (char *_Restrict, size_t, const char *_Restrict,
                             const struct tm *_Restrict);
errno_t       asctime_s     (char *, rsize_t, const struct tm *);
errno_t       ctime_s       (char *, rsize_t, const time_t *);
struct tm *   gmtime_s      (const time_t *_Restrict, struct tm *_Restrict);
struct tm *   localtime_s   (const time_t *_Restrict, struct tm *_Restrict);
void          tzset         (void);
char *        asctime_r     (const struct tm *, char *);
char *        ctime_r       (const time_t *, char *);
struct tm *   gmtime_r      (const time_t *, struct tm *);
struct tm *   localtime_r   (const time_t *, struct tm *);

char *        tzname[2];
```

time.h continues to include **clockLib.h** and **timerLib.h**. These can be suppressed through the **_ISO_C_SOURCE** feature test macro.

In addition, the following definitions can also be suppressed though the **_ISO_C_SOURCE** feature test macro:

```
void             tzset        (void);
char *           asctime_r    (const struct tm *, char *);
char *           ctime_r      (const time_t *, char *);
struct tm *      gmtime_r     (const time_t *, struct tm *);
```

```
struct tm *       localtime_r  (const time_t *, struct tm *);
char *            tzname[2];
```

The following X/Open Source Interface extensions related to tzset( ) are also available but can be suppressed through the **_POSIX_C_SOURCE** feature test macro:

```
long            timezone;
int             daylight;
```

Summary of Changes to time.h on page 40

5.1. Summary of Changes to time.h

- The *daylight* and *timezone* variable definitions are always provided unless the **_POSIX_C_SOURCE** feature test macro is defined.
- The **_VXWORKS_COMPATIBILITY_MODE** macro is no longer supported, i.e. the VxWorks specific variants of asctimer_r( ), ctime_r( ), gmtime_r( ), and localtime_r( ) are no longer supported.

Parent topic: Date and Time Interfaces: time.h on page 37

6. General Utility Interfaces: stdlib.h

Previously, the following interfaces were provided if the **_POSIX_C_SOURCE**, **_POSIX_AEP_RT_CONTROLLER_C_SOURCE**, or **_XOPEN_SOURCE** feature test macros were defined by the including source:

```
long            a64l        (const char *);
char *          l64a        (long);
double          drand48     (void);
double          erand48     (unsigned short *);
long            jrand48     (unsigned short *);
void            lcong48     (unsigned short *);
long            lrand48     (void);
long            mrand48     (void);
long            nrand48     (unsigned short *);
void            srand48     (long);
unsigned short *seed48      (unsigned short *);
```

Note: Any compatibility issues related to environment variable related interfaces are covered in Environment Variables on page 31

Also, the following VxWorks specific interfaces were provided but only if the **_POSIX_C_SOURCE**, **_XOPEN_SOURCE**, and **_POSIX_AEP_RT_CONTROLLER_C_SOURCE** feature test macros were not defined:

```
long             irand48 (unsigned short);
long             krand48 (unsigned short *, unsigned short);
```

As a result of the **C11** standard library upgrade, the following interfaces are no longer provided:

```
long            a64l        (const char *);
char *          l64a        (long);
long            irand48     (unsigned short);
long            krand48     (unsigned short *, unsigned short);
```

Also, the following interfaces are unconditionally provided:

```
double          drand48     (void);
double          erand48     (unsigned short *);
long            jrand48     (unsigned short *);
void            lcong48     (unsigned short *);
long            lrand48     (void);
long            mrand48     (void);
long            nrand48     (unsigned short *);
void            srand48     (long);
unsigned short *seed48      (unsigned short *);
```

7. Time zone: TIMEZONE and TZ Environment Variables

As a result of the **C11** standard library upgrade, the *POSIX.1-2008 TZ* and the *TIMEZZONE* environment variables are supported. If both the *TZ* and *TIMEZONE* environment variables have been set, then the value of the TZ environment variable shall take precedence.

The default time zone setting, i.e. when both the TZ and TIMEZONE environment variables are not set, is :

```
:UTC:UTC:0000
```

The *POSIX.1-2008* specified tzset( ) interface is provided to use the value of the *TZ* or *TIMEZONE* environment variables for usage by the ctime( ) , localtime( ) , mktime( ) , and strftime( ) interfaces.

The method of loading time zone information from a file is no longer supported.

Note: The **TIMEZONE_ENV_VAR** parameter of the **INCLUDE_DEFAULT_TIMEZONE** component can be used to change the default time zone setting when both the *TZ* and *TIMEZONE* environment variables are not set.

8. Input and Output Interfaces: stdio.h

Previously, the following interfaces were provided through **stdio.h**:

```
char *   ctermid             (char *);
void     flockfile           (FILE *);
int      fseeko              (FILE *, off_t, int);
off_t    ftello              (FILE *);
int      ftrylockfile        (FILE *);
void     funlockfile         (FILE *);
int      getc_unlocked       (FILE *);
int      getchar_unlocked    (void);
int      pclose              (FILE *);
FILE *   popen               (const char *, const char *);
```

```
int      putc_unlocked       (int, FILE *);
int      putchar_unlocked    (int);
```

Note: Although the ctermid( ), pclose( ), and popen( ) function prototypes were provided in **stdio.h**, implementations for these function were not provided.

The following X/Open System Interface definition was also provided in stdio.h when either the **_XOPEN_SOURCE** or **_POSIX_AEP_RT_CONTROLLER_C_SOURCE** feature test macros were defined by the including source code:

```
char *  tempnam         (const char *, const char *);
```

As a result of the **C11** standard library upgrade, only the following function prototypes are available through **stdio.h**, but can be suppressed through the **_ISO_C_SOURCE** feature test macro:

```
int       fseeko             (FILE *, off_t, int);
off_t     ftello             (FILE *);
void      flockfile          (FILE *);
int       ftrylockfile       (FILE *);
void      funlockfile        (FILE *);
int       getc_unlocked      (FILE *);
int       getchar_unlocked   (void);
int       putc_unlocked      (int, FILE *);
int       putchar_unlocked   (int);
```

9. Non-Local Jumps: setjmp.h

The following function prototypes are available through **setjmp.h** and can be suppressed through the **_ISO_C_SOURCE** feature test macro:

```
int      sigsetjmp      (sigjmp_buf,int);
void     siglongjmp     (sigjmp_buf, int);
```

10. Localization

In VxWorks, **C** standard localization is the default setting. However, you can use the **C11** standard library for many European or American locales in your applications. For the complete list, see the *installDir* **/vxworks-7/pkgs_v2/os/lang_lib/libc/libc_std/locales** directory. Each file in this directory defines a European or American locale (for example, the **en_CA** file is used for the English language locale in Canada). The directory also includes a **readme.txt** file that includes code examples.

Procedure

1. On host machine, concatenate the locales you want to use in a single file.

   To use all the locales:

   ```
   $ cat POSIX *_* > /mydir/locales.txt
   ```

2. (Optional) If your **locales.txt** file has DOS newline characters (**\r\n**), replace them with the Unix newline character (**\n**).

```
$ dos2unix /mydir/locales.txt
```

3. Copy the **locales.txt** file to your target storage (for example, **/romfs/locales.txt**).
4. In your RTP on the target, set the environment variable *LOCFILE* to point to the **locales.txt** file.

   If **locales.txt** is located at **/romfs/locales.txt**:

```
putenv("LOCFILE=/romfs/locales.txt")
```

   -or-

```
setenv("LOCFILE", "/romfs/locales.txt", 1)  /* the 3rd argument 1 means overwriting if environm
ent variable LOCFILE already exists  */
```

5. Set the desired locale for your RTP using the setlocale command.

```
$ setlocale(LC_ALL, "de_AT")
```

8. RTP ENVIRONMENT: C++14/C++17 COMPATIBILITY

1. RTP Environment: C++14/C++17 Compatibility

Previously, the RTP environment provided two options for **C++** :

- **C++03** libraries similar (but not identical) to those used in the kernel environment (see Kernel Environment: C++14/C++17 Compatibility on page 27)
- **C++11** libraries (only available with GCC and ICC)

In this release, the RTP environment provides libraries conforming to the **C++ 17** standard, ISO/IEC 14882:2017.

Application code will continue to be compiled as C++14 by default. To use C++17 in an application or layer, add the following compiler option to the build tool flags:

```
-std=c++17
```

2. Wrappers Supporting Interfaces using FUNCPTR

As a result of the **C++14/C++17** standard library upgrade, the series of **C++** inline wrappers that support VxWorks native interfaces using the *FUNCPTR* type definition have been removed. These wrappers were introduced in VxWorks 6.x to maintain backwards compatibility for **C++** applications that used VxWorks native interfaces that were changed to use fully-qualified function pointers instead of *FUNCPTR* , *VOIDFUNCPTR* , *DBLFUNCPTR* , and *FLTFUNCPTR* .

The following are the **C++** inline wrappers that have been deleted:

- **dllLib.h**

```
inline DL_NODE * dllEach (DL_LIST *, FUNCPTR, _Vx_usr_arg_t)
```

- **hashLib.h**

```
inline HASH_NODE * hashTblEach(HASH_ID,FUNCPTR,_Vx_usr_arg_t)
```

- **sllLib.h**

```
inline SL_NODE * sllEach (SL_LIST *, FUNCPTR ,_Vx_usr_arg_t)
```

- **symLib.h**

```
inline SYMBOL_ID symEach (SYMTAB_ID, FUNCPTR, _Vx_usr_arg_t)
```

- **taskLib.h**

```
inline STATUS taskCreateHookAdd         (FUNCPTR createHook)
inline STATUS taskCreateHookDelete      (FUNCPTR createHook)
inline STATUS taskDeleteHookAdd         (FUNCPTR deleteHook)
inline STATUS taskDeleteHookDelete      (FUNCPTR deleteHook)
```


WNDRVR

3. C++ Exception Handling

See C++ Exception Handling on page 30 for more information.

4. RTP C++17 Compatibility

Users wishing to migrate to C++17 should be aware of a few ways in which C++17 is backward incompatible with C++14.

For a full list of changes see:

https://isocpp.org/files/papers/p0636r0.html

In particular dynamic exception specifications are no longer supported. Since C++11 compile-time checked "noexcept" specifications have been preferred to dynamically checked exception "throw(A, B, ...)" specifications. In C++17, the throw(A, B, ...) syntax is no longer supported. However **throw()** with no arguments is still allowed but is now an alias for "noexcept". Support for **throw()** will be removed in C++20.

The VxWorks implementation of C++17 has the following limitation:

- Parallel algorithms (for example **par_reduce**) are not yet supported.

WNDRVR

9. LLVM TOOLCHAIN COMPATIBILITY

1. Overview

Previously, for the Intel processor architecture:

- A Wind River customized version of the GNU compiler was used to compile the VxWorks kernel libraries, and
- A Wind River customized version of the GNU compiler, the Wind River Compiler (only for 32-bit), or the Intel compiler was available to generate **VIPs**, **DKMs**, **RTPs**, static/shared **RTP** libraries.

Previously for the ARM processor architecture:

- The Wind River Compiler (only for 32-bit only) and the LLVM compiler (only for 64-bit) was used to compile the VxWorks kernel libraries, and
- A Wind River customized version of the GNU compiler was used to generate **VIPs, DKMs**, **RTPs**, static/shared **RTP** libraries.

Starting with the VxWorks 7 SR0600 release, the only supported toolchain for the Intel, ARM, and RISC-V processor architectures is LLVM. The toolchain consists of:

```
Compiler:       Clang
Assembler:      Clang built-in assembler
Linker:         ld<arch> (GNU linker)
```

The following are some notes on toolchain-related terminology:

- **Clang** is a **C/C++** compiler built using **LLVM**
- **LLVM** is a modern, highly optimizing compiler technology

Clang is highly compatible with **GCC**, and code that compiles with **GCC** usually compiles without modification with **Clang**. The following sections detail the few cases in which changes may be required. While the basic usage model of the toolchain remains similar, the compiler tools (ctoa and etoa) have been completely replaced with an **LLVM**-based compiler front-end (Clang). **LLVM** is a modern, fast, highly optimizing compiler that supports the latest **C** and **C++** standards, as well as many features that allow it to be highly compatible with the GNU compiler. This guide is intended to help existing Diab customers to migrate to the new **LLVM**-based toolchain.

2. Assembly Code

Assembly code, whether embedded in **C/C++** code, or appearing in separate assembly (.s) files, is assembled using the Clang "built-in" assembler. This assembler is highly compatible with the GNU assembler (as<*arch*>), but in some cases may be more strict.

In particular on Intel targets, an explicit size suffix is required for instructions that reference memory. For example:

```
cmp $0,(%eax)
```

This is accepted by the GNU assembler, which treats it as implicitly having an "l" suffix (meaning 4 byte memory access). However, Clang gives an error:

```
error: ambiguous instructions require an explicit suffix (could be 'cmpb', 'cmpw', 'cmpl', or 'cmp
q')
        cmp $0,(%eax)
```

The fix is to make the intent clear by adding an explicit suffix. For existing code this usually just means adding an "l" suffix as shown in the following:

```
cmpl $0,(%eax)
```

This assembly is highly compatible with the GNU assembler. Code written for the Diab assembler may need to be modified to use GNU syntax.

3. Optimization

This section describes optimization options.

For information, see Optimization Options on page 52.

Disabling Optimization for a Specific Function on page 47 For debug-ability or correctness, some low-level functions may need to be compiled with all optimizations disabled. Applications compiled with GCC could achieve this using the following construct:
Optimization Options on page 47 While Clang accepts most **GCC** options, it is based on a completely different optimizing technology. Therefore the same options used on Clang and **GCC** for a particular application may produce quite different performance/footprint trade-offs. If the default options used in VxWorks 7 do not produce the desired results, consider experimenting with other optimization levels.

3.1. Disabling Optimization for a Specific Function

For debug-ability or correctness, some low-level functions may need to be compiled with all optimizations disabled. Applications compiled with GCC could achieve this using the following construct:

```
__attribute__((optimize("O0")))
```

To achieve the same effect with Clang, use the following construct:

```
__attribute__((optnone))
```

The following is an example usage on the function named myFunction( ):

```
__attribute__((optnone)) int myFunction
    (
    int myInteger
    )
    {
    ...
    }
```

Parent topic: Optimization on page 47

3.2. Optimization Options

While Clang accepts most **GCC** options, it is based on a completely different optimizing technology. Therefore the same options used on Clang and **GCC** for a particular application may produce quite different performance/footprint trade-offs. If the default options used in VxWorks 7 do not produce the desired results, consider experimenting with other optimization levels.

The default optimization level in VxWorks 7 is -O2. Note that the default optimization level used with GCC was also –O2. Given the different optimizing technology, the code generated from LLVM –O2 will be different than GCC –O2.

The following description of the Clang optimization related options was taken from:

http://releases.llvm.org/6.0.0/tools/clang/docs/CommandGuide/clang.html#code-generation-options

- -O0 No optimization
- -O1 Somewhere between -O0 and -O2.
- -O2 Moderate level of optimization which enables most optimizations (<==VxWorks default)
- -O3 Like -O2, except that it enables optimizations that take longer to perform or that may generate larger code (in an attempt to make the program run faster).
- -Ofast Enables all the optimizations from -O3 along with other aggressive optimizations that may violate strict compliance with language standards.
- -Os Like -O2 with extra optimizations to reduce code size.
- -Oz Like -Os (and thus -O2), but reduces code size further.
- -Og Like -O1. In future versions, this option might disable different optimizations in order to improve debug-ability.
- -O Equivalent to -O2.

See also Optimization Options on page 52

Parent topic: Optimization on page 47

4. The –fvolatile Option

This option causes all memory references through pointers to be volatile. This option is not supported with Clang. Also note that starting with the "VxWorks 7 Release Notes, Release SR0490," the following item was listed under "Deprecation Notices":

–fvolatile

The –fvolatile option of the GNU compiler is deprecated. Update your source code to use the volatile type qualifier on pointer declarations as required. The –fvolatile option of the GNU compiler is deprecated. Update your source code to use the volatile type qualifier on pointer declarations as required.

5. Migrating from Diab to LLVM

Attribute Specifier Comparison on page 49 This section shows the **LLVM** equivalents of Diab attribute specifiers..
Component Comparison on page 49 This section compares compiler components between **Diab** and **LLVM**.
Keyword Comparison on page 50 This section lists **Diab** keywords and their **LLVM** equivalents, if any.
Option Comparison on page 51 This section lists Diab options and their equivalents in **LLVM**, if any. For options that affect warning messages, see Warning Messages.
Comparisons of Other Additions to ANSI C and C++ on page 53 This section lists other features of the Diab compiler that are additions to the **ANSI C** and **C++** standard.
Output Comparison on page 54 The following are differences in output between **Diab** and **LLVM**:
Pragma Comparison on page 54 This section lists **Diab** pragmas and their **LLVM** equivalents.
Predefined Preprocessor Macro Comparison on page 55 This section lists predefined preprocessor macros in **Diab** and indicates whether they're also defined in **LLVM**.
Preprocessor Directive Comparison on page 57 This section lists **Diab** preprocessor directives and their **LLVM** equivalents, if any.

Warning Message Comparison on page 58 The options listed in this section affect warning messages.

5.1. Attribute Specifier Comparison

This section shows the **LLVM** equivalents of Diab attribute specifiers..

Diab Attribute Specifier	LLVM Equivalent
absolute (**C** only)	no equivalent
aligned(n)	no modification required
always_inline	no modification required
constructor(n)	no modification required
deprecate (**C** only) **deprecated**(string) (**C** only)	no modification required
destructor **destructor(n)**	no modification required
flatten	no modification required
noinline	no modification required
noreturn **no_return**	use **noreturn** **no_return** is not supported
no_side_effects	equivalent to **pure** attribute
pure **pure_function**	equivalent is **const** attribute
packed	no modification required
section(*name*)	no modification required
use_frame_pointer	The default for Clang is to use frame pointers for all functions. Using the -fomit-frame-pointer option tells the compiler to not use frame pointers for functions. -fomit-frame-pointer is enabled when using **-O**, **-O2, -O3**, or **–Os**.

Parent topic: Migrating from Diab to LLVM on page 48

5.2. Component Comparison

This section compares compiler components between **Diab** and **LLVM**.

Diab Component	LLVM Component	Description
ctoa **etoa**	**clang**	compiler
dar	**ar**<*arch*>	archiver
das	**clang**	clang integrated assembler
dcc **dplus**	**clang** **clang**	driver
ddump	**nm**<*arch*> **objcopy**<*arch*> **objdump**<*arch*> **readelf**<*arch*> **size**<*arch*>	binary utilities
dld	**ld**<*arch*>	linker

Parent topic: Migrating from Diab to LLVM on page 48

5.3. Keyword Comparison

This section lists **Diab** keywords and their **LLVM** equivalents, if any.

Diab Keyword	LLVM Equivalent
__asm **asm**	Use **__asm** and **asm** for embedding inline assembly code into a **C/C++** function using GNU inline assembly syntax. To include an entire assembly function inside a **C** or **C++** file, use **__attribute__((naked))**.
__attribute__	no modification required
__bit	no equivalent
extended (**C** only)	no equivalent
__inline__ **inline**	no modification required Use of these keywords depends on the language standard you are using, as specified by the -std option. For example, inline is not supported when using -std=c90 (which is equivalent to -ansi).
long long	no modification required
__packed__ **packed**	**use __attribute__((packed))**

Diab Keyword	LLVM Equivalent
__typeof__ (**C** only)	no modification required

Parent topic: Migrating from Diab to LLVM on page 48

5.4. Option Comparison

This section lists Diab options and their equivalents in **LLVM**, if any. For options that affect warning messages, see Warning Messages.

Non-Optimization Options on page 51 This section lists **Diab** and equivalent **LLVM** options (non-optimization)
Optimization Options on page 52 This section lists Diab optimization options and their equivalents in LLVM.

Parent topic: Migrating from Diab to LLVM on page 48

Non-Optimization Options

This section lists **Diab** and equivalent **LLVM** options (non-optimization)

Diab Option	LLVM Option	Description
-c	-c	Stop after assembly, produce object file
-E	-E	Stop after preprocessor, write source to standard output by default. With **LLVM**, use -E-*ofile* to write the output to file (not honored with Diab).
-I *file* .h	-include *file* .h	Include *file* .h before processing any other source files.
-o	-o	Output to the given file instead of the default
-P	-P	Causes preprocessed output without line markers to be written to a .i file with Diab. In **LLVM,** -P is simply a modifier to -E. The only effect is to remove line markers from the output.
-S	-S	Stop after compilation, produce assembly file
-Xdialect-c89	-std=c89	Follow the **C89** standard for **C**
-Xdialect-c99	-std=c99	Follow the **C99** standard for **C**
-Xdialect-c11	-std=c11	Follow the **C 2011** standard
-Xdialect-ansi -Xansi	-ansi	Follow the **ANSI C** standard, with some additions

Diab Option	LLVM Option	Description
-Xdialect-strict-ansi -Xstrict-ansi	-ansi -pedantic-errors	Strictly follow the **ANSI C** and **C++** standards
-Xdialect-c++03	-std=c++03	Support the **C++ 2003** standard
-Xdialect-c++11	-std=c++11	Support the **C++ 2011** standard
-Xdialect-c++14	-std=c++14	Support the **C++ 2014** standard
-Xmake-dependency -Xmake-dependency=1	-M	Show all required include files. Writes to **stdout** by default
-Xmake-dependency=2	-MMD	Show required user header files
-Xmake-dependency=4	-MD	Show all required include files. Writes to *filename*.d by default
-Xmake-dependency=8	-MP	Show all required include files but output a phony target for each dependency other than the main file.
-Xmake-dependency-savefile=*filename*	-MF*filename*	Specify dependency output file
-Xpass-source	-g-Xassembly-listing	With Diab, -Xpass-source is usually used together with -Xkeep-assembly-file and outputs the source as comments in the generated assembly language code. With **LLVM**, using -g together with -Xassembly-listing outputs the source as comments in the generated listing file.
-Xpreprocess-assembly	-Xpreprocess-assembly	Invoke **C** preprocessor on assembly files before running the assembler
-Xsection-split	-ffunction-sections -fdata-sections	Place each function and each variable in its own section class (CODE for functions, DATA for variables). The names of the generated sections are in the form of *base-section-name.symbol*.

Parent topic: Option Comparison on page 51

Optimization Options

This section lists Diab optimization options and their equivalents in LLVM.

See also Optimization Options on page 47

Diab 5x Option	LLVM Option	Description
-Ox	-O	Perform different levels of optimization. The value x sets the level of effort to optimize for speed (-O, -O0, -O1, -O2, -O3). -O4 is deprecated in **LLVM** and will issue a warning
-XO	-O2 or -O3	Perform extra optimizations
-Os -Xsize-opt	-Os -Oz	Optimize for size rather than speed when there is a choice. In LLVM this is a separate orthogonal optimization level, not an additional dimension of optimization as it is in Diab, i.e., -O2–Os just means –Os.
-g*n*	-g	Generate debugging information. In Diab, -gn enables different levels of debugging information (for example –g1, -g2, –g3). LLVM only supports –g.

Parent topic: Option Comparison on page 51

5.5. Comparisons of Other Additions to ANSI C and C++

This section lists other features of the Diab compiler that are additions to the **ANSI C** and **C++** standard.

Diab Feature	LLVM Feature
support for 64-bit bit-fields	yes
C++ comments	yes
dynamic memory allocation with alloca( )	Use **__builtin_alloca**
binary representation of data	yes
__ERROR__ function	no
sizeof extension	no
statement expressions	yes
variadic macros	yes
static initialization of flexible array members	yes
initialization of a range of array members with the same value	yes

WNDRVR

Diab Feature	LLVM Feature
zero-sized structures and unions as members of structures	yes

Parent topic: Migrating from Diab to LLVM on page 48

5.6. Output Comparison

The following are differences in output between **Diab** and **LLVM**:

- The format of error messages and warnings differs between **Diab** and **LLVM**
- In Diab 5.9.x, debug information generated with -g defaults to **DWARF 2**. In **LLVM**, debug information generated with -g defaults to **DWARF 4**. To generate debug information conforming to a different version of **DWARF**, use -gdwarf-*<DWARF-version>*; e.g., -gdwarf-2 will generate **DWARF 2**.

For options that affect warning messages, see Warning Message Comparison on page 58

Parent topic: Migrating from Diab to LLVM on page 48

5.7. Pragma Comparison

This section lists **Diab** pragmas and their **LLVM** equivalents.

Diab Pragma	LLVM Equivalent
align	use **__attribute__((aligned(***x***)))**
align_functions	use **__attribute__((aligned(***x***)))**
align_variables	use **__attribute__((aligned(***x***)))**
always_inline	use **__attribute__((always_inline))**
contract	no modification required
error	equivalent pragma: **#pragma GCC error** "*error message*"
flatten	use **__attribute__((flatten))**
global_register	not supported
hdrstop	not supported
ident	not supported
info	not supported
inline	use **__attribute__((always_inline))**

Diab Pragma	LLVM Equivalent
interrupt	use **__attribute__((interrupt))**
no_alias	not supported
noinline	use **__attribute__((noinline))**
no_pch	not supported
no_return	use **__attribute__((noreturn))**
no_side_effects	use **__attribute__((pure))**
option	not supported
pack	use **__attribute__((packed))**
pure_function	use **__attribute__((const))**
section	use **__attribute__((section(**"*name* **"))**
unroll	equivalent pragma: #**pragma clang loop unroll(enable | full | disable)** #**pragma clang loop unroll_count**(*unrollFactor*)
use_section	not supported
warning	equivalent pragma: #**pragma GCC warning** *"warning_message* "
weak	use **__attribute__((weak))**

Parent topic: Migrating from Diab to LLVM on page 48

5.8. Predefined Preprocessor Macro Comparison

This section lists predefined preprocessor macros in **Diab** and indicates whether they're also defined in **LLVM**.

Macro	LLVM
__arm	yes
__BIG_ENDIAN__	yes
__bool	no
__CHAR_UNSIGNED__	yes

Macro	LLVM
__cplusplus	yes; however, does not expand to 1 but to the C++ version number
__DATE__	yes
__DCC__	yes
__DCC_LLVM__	yes
__DCPLUSPLUS__	no
_DIAB_TOOL	yes
__EABI__	yes
__ETOA__	no
__ETOA_IMPLICIT_USING_STD	no
__ETOA_NAMESPACES	no
__EXCEPTIONS	yes
__FILE__	yes
__FUNCTION__	yes
__LDBL_	no
__LITTLE_ENDIAN__	yes
__LINE__	yes
__lint	no
__nofp	no
__PRETTY_FUNCTION__	yes
_RTTI	equivalent macro in **LLVM: __GXX_RTTI**
__SIGNED_CHARS__	no
__softfp	no
__STDC__	yes
__STRICT_ANSI__	yes

Macro	LLVM
__TIME__	yes
__THUMB__	no
__THUMB2__	yes (THUMB 2 targets only)
__VERSION__ **__VERSION_NUMBER__**	no

Parent topic: Migrating from Diab to LLVM on page 48

5.9. Preprocessor Directive Comparison

This section lists **Diab** preprocessor directives and their **LLVM** equivalents, if any.

Diab Preprocessor Directive	LLVM
#assert **#unassert**	not supported
#error	no modification required
#ident (C only)	not supported
#import	no modification required
#info **#inform** **#informing**	not supported
#warn **#warning**	use **#warning**

Syntax Comparison on page 57 There are notable syntax differences between **Diab** and **LLVM**. For example, with **Diab**, many driver options, and all compiler options, began with -X. With **LLVM**, compiler options use the Clang naming conventions (which are themselves based on **GCC**):

Parent topic: Migrating from Diab to LLVM on page 48

Syntax Comparison

There are notable syntax differences between **Diab** and **LLVM**. For example, with **Diab**, many driver options, and all compiler options, began with -X. With **LLVM**, compiler options use the Clang naming conventions (which are themselves based on **GCC**):

- Target independent options are named -f<*option*>.
- Target dependent options are named -m<*option*>.
- Many options can be disabled using the no- options: -fno-<*option*> and -mno<*option*>.

Parent topic: Preprocessor Directive Comparison on page 57

5.10. Warning Message Comparison

The options listed in this section affect warning messages.

Action	Diab	LLVM
disable the specified warning	ei<*number*>	-Wno<*warning*>
enable the specified warning	ew<*number*>	-W<*warning*>
turn the specified warning into an error	ee<*number*>	-Wfatal-errors
cause the compiler to abort on the first error	ef<*number*>	-Wfatal-errors

Parent topic: Migrating from Diab to LLVM on page 48

6. Undefined Behavior in C and C++

The ISO/IEC 9899:2011 specification defines "undefined behavior" as:

"behavior, upon use of a nonportable or erroneous program construct or of erroneous data, for which this International Standard imposes no requirements.

NOTE Possible undefined behavior ranges from ignoring the situation completely with unpredictable results, to behaving during translation or program execution in a documented manner characteristic of the environment (with or without the issuance of a diagnostic message), to terminating a translation or execution (with the issuance of a diagnostic message)."

Examples of undefined behaviors in the **C** language specification include dereferencing a NULL pointer, integer overflow, omitting the return statement in a function that is supposed to return a value, and divide by zero. Many programmers have certain expectations of the behavior when dereferencing a NULL pointer, for example. The expectation is that a "page fault" exception will be raised at runtime when a NULL pointer is dereferenced (assuming that the memory page at address 0 is not mapped); however, the **C** language specification does not define any specific behavior for dereferencing a NULL pointer, and thus the compiler or runtime environment is free to implement any behavior for that operation.

Undefined behavior allows a compiler to generate more efficient code in certain situations. For example, in a loop where a signed integer is incremented, the **C/C++** compiler does not need to generate code to detect/handle the situation where the variable overflows.

The optimizer in Clang is more aggressive with regard to taking advantage of undefined behavior. Thus code that appeared to behave "correctly" when compiling with GCC or the Wind River compiler may not behave identically when compiled with Clang. Moreover, Clang may not issue a warning message when compiling code that utilizes an undefined operation. The "**__attribute__((optnone))**" construct can be used to defeat the optimizer on a given function, and this might result in the identical behavior that occurred when compiling with GCC or Wind River compiler. However, the resultant behavior is "undefined" and thus it's best to update the source code to eliminate the usage of undefined operations

In some cases Clang might deliberately insert a *trap* or *undefined* instruction where undefined behavior would arise. For example, when optimization is disabled, Clang inserts a trap instruction to replace a missing return statement in a function (except for main( )) that is supposed to return a value.

Adding a non-zero offset to a NULL pointer is undefined behavior in C and C++. In some cases the Clang compiler may take advantage of that to make the following transformation:

```
if (ptr + offset) {…}  => if (ptr) {…}
```

In the (in practice, highly unusual) case that a pointer type is used to hold an arbitrary integer, the above transformation may lead to unexpected behavior. To avoid this, only use pointers to represent addresses.

10. GNU TOOL COMPATIBILITY

1. Overview

Previously, for the PowerPC processor architecture:

- The Wind River Compiler was used to compile the VxWorks kernel libraries, and
- A Wind River customized version of the **GNU** compiler was available to generate **VIP**s, **DKM**s, **RTP**s, static/shared **RTP** libraries.

Starting with the VxWorks 7 SR0600 release, a non-Wind River specific version of the GNU compiler is used: gcc 8.1. For more information, see https://gcc.gnu.org/onlinedocs/8.1.0/.

The remaining sections will highlight differences with this compiler and its impacts to VxWorks 7 PowerPC architecture support.

2. The –mregnames option

Previously, the compilation of user-space **C** and **C++** source files (with the GNU compiler) specified the PowerPC specific compiler option: -Wa,-mregnames. The "-Wa," prefix simply causes the subsequent option, i.e. –mregnames, to be passed to the assembler. This option allowed embedded assembler code to reference registers with the labels **r3**, **f1**, **sp**, for example. However, this option had the side-effect that user-space **C/C++** code could not use those same labels as identifiers, e.g. function names, variable names, etc.

Thus starting with the VxWorks SR0600 release, the -Wa,-mregnames option is no longer specified when compiling **C** and **C++** source files. This change requires modifications to PowerPC inline assembly code to replace register labels with register numbers. For example, the following code:

```
__WRS_ASM("addi r7,r3,r4");
```

Needs to change to the following:

```
__WRS_ASM("addi 7,3,4");
```

Alternatively, the Makefile for the directory containing the **C/C++** source code using embedded assembly code referencing the register labels, e.g. sample.c, can be updated to include the following construct:

```
CFLAGS_sample.o += $(OPTION_ASM_FLAGS)
```

Assembly source files need not be changed; they can still reference registers using the labels **r3**, **f1**, **sp**, for example.

3. The –fno-implicit-fp option

Previously, the compilation of **C** and **C++** source files specified the –fno-implicit-fp option with the GNU compiler. The specification of this option prevented the compiler from issuing "floating point" instructions, and instructions which used floating point registers unless the source code explicitly operated on a floating point variable.

This option was specified to ensure a task context, or an ISR, did not unexpectedly utilize floating point registers and thus potentially corrupt the state of another task (or ISR) using floating point registers. An "application" needs to explicitly specify the VX_FP_TASK

option bit when creating/spawning a VxWorks task, thus the unexpected utilization of a floating point register by the compiler can easily result in corrupted floating point register state.

> Note: The term "floating point" is used to denote ANY non-scalar-integer register or instruction. This includes x87 FPU, MMX, SSE, and AVX instructions and their corresponding registers for the Intel architecture. Similarly this includes the NEON and VFP units for the ARM architecture, and the FPU, SPE, and AltiVec units for the PowerPC architecture.

In addition, limiting the usage of floating point registers to the select task (and ISR) contexts that explicitly utilize floating point variables (or invoke functions that accept floating point typed arguments) helps to maximize real-time performance w.r.t. context switch latency and interrupt latency. In other words, floating point registers only need to be saved and restored for the select task contexts that explicitly utilize floating point variables.

Modern compilers extensively employ optimizations using floating point resources, thus if the –fno-implicit-fp option is not specified, most task context switches and ISR entry/exit stubs would require that the floating point context be saved and restored. This would increase task switch latency for most, if not all, tasks and increase interrupt latency for most, if not all, ISRs.

Specifying the –fno-implicit-fp option has the disadvantage that the compiler is prevented from employing certain optimizations such as using a "wide" floating point register to quickly copy memory contents, or auto-vectorization. As compiler technology advances, the techniques for leveraging the floating point resource on a processor increase. Thus specifying the –fno-implicit-fp deprives applications of (throughput) performance that is available by the compiler employing auto-vectorization, for example.

Starting with VxWorks 7 SR0600, a non-Wind River specific version of the GNU compiler is used: gcc 8.1. The –fno-implicit-fp option is no longer supported by this compiler, nor will it likely ever be supported in future versions of the GNU compiler.

As mentioned previously, allowing the compiler to implicitly leverage the floating point resources of the processor results in improved performance but at the cost of real-time performance in terms of task switch and interrupt latency. Thus VxWorks 7 SR0600 for the PowerPC architecture introduces a pair of new floating point modes named **ksoft** and **khard**. The **hard** floating point mode is no longer supported for the PowerPC architecture.

The baseline capabilities for the new **ksoft** and **khard** floating point (FP) modes include:

- User-space code can freely use hardware floating point operations including implicit usage by compiler optimizations.
- Kernel code can freely use floating point at the **C/C++** language level (although the performance of such code will be significantly greater in **khard** mode than in **ksoft**).

The following describes the differences between the 3 floating point modes available with the PowerPC architecture:

- **khard** : hardware floating point operations will be utilized in both the kernel and user-space. Given that the –fno-implicit-fp option no longer exists, the **khard** mode implies that the compiler will leverage the processors floating point resources even if kernel or user-space source code doesn't explicitly utilize floating point variables.

  These semantics differ from the previously offered **hard** mode where the –fno-implicit-fp compiler option was specified, i.e. the **hard** mode was used to specify the usage of hardware floating point operations but implicit floating point operations were suppressed. Given the significant difference in semantics, the **khard** mode was introduced rather than merely repurposing the existing **hard** mode to prevent any confusion.
- **ksoft** : hardware floating point operations will be utilized in user-space, but software floating point operations will be used in the kernel. The compiler will leverage the processors floating point resources even if user-space source code doesn't explicitly utilize floating point variables. However, the compilation of kernel source code shall only use integer registers and instructions.
- **soft** : software floating point operations will be utilized in both the kernel and user-space. All operations in both the kernel and user-space will be limited to integer registers and instructions.

The **khard**, **soft**, and **ksoft** FP modes are provided for all PowerPC CPUs supported in VxWorks 7.

The availability of the **khard**, **ksoft**, and **soft** floating point modes allows a system designer to choose the appropriate mode based on performance requirements:

- The **khard** FP mode should be selected to maximize throughput performance in both the kernel and user-space but at the expense of increasing task switch latency for all tasks and ISR contexts. When the **khard** FP mode is selected, the VX_FP_TASK

option bit is implicitly set by the operating system, for both kernel and RTP tasks, and implicit fppSave()/fppRestore() operations are performed when invoking an ISR.

- The **ksoft** FP mode should be selected to maximize throughput performance in user-space and to maximize real-time performance in the kernel w.r.t. kernel task context switch latency and ISR latency. When the **ksoft** FP mode is selected, the VX_FP_TASK option bit is implicitly set by the operating system only for RTP tasks, and implicit fppSave()/fppRestore() operations are not performed when invoking an ISR.
- The **soft** FP mode should be selected to maximize real-time performance in both the kernel and user-space with regards to task switch latency for both kernel and RTP tasks, and ISR latency. When the **soft** FP mode is selected, the VX_FP_TASK option bit is **not** implicitly set by the operating system for any task, and implicit fppSave()/fppRestore() operations are **not** performed when invoking an ISR.

As noted in the descriptions for **khard**, **ksoft**, and **soft**, the operating system will implicitly set the VX_FP_TASK task option bit as appropriate. An "application" can elect to explicitly set the VX_FP_TASK task option as needed. For example, in **a ksoft** configured system, a given kernel task can execute code written in assembly language which utilizes floating point operations, or can execute **C/C++** code that was compiled with the –mhard-float compiler option. In such cases, the task must be created by explicitly specifying the VX_FP_TASK option bit when creating the kernel task.

Similarly, in a **ksoft** configured system, for example, an ISR can execute code written in assembly utilizes floating point operations, or can execute **C/C++** code that was compiled with the –mhard-float compiler option. In such cases, the ISR must explicitly call fppSave() as the first operation in the handler, and then call fppRestore() as the very last operation of the handler.

An application must not clear the VX_FP_TASK option bit for tasks where the operating system has implicitly set the VX_FP_TASK task option bit when creating the task.

As mentioned above, the **hard** floating point mode is no longer supported for the PowerPC architecture due to the absence of the –fno-implicit-fp compiler option with non-Wind River specific versions of the GNU compiler. Nonetheless, the semantics of the **hard** floating point mode introduced the possibility of a task inadvertently utilizing hardware floating point operations. Although hardware floating point operations are only used in the generated code if the **C/C++** source code explicitly uses floating point variables, the onus was on the "application" to correctly specify the VX_FP_TASK option bit when creating a task. Thus there existed the possibility of a task being created without the VX_FP_TASK option bit being specified, that inadvertently executed a function that utilized hardware floating point operations.

The **khard** floating point mode avoids this potential situation since the operating system implicitly sets the VX_FP_TASK option bit for all tasks, and implicitly invokes fppSave()/fppRestore() when invoking an ISR.